CHAPTER OUTLINE

3.1 Disks
- 3.1.1 The Organization of Disks
- 3.1.2 Estimating Capacities and Space Needs
- 3.1.3 Organizing Tracks by Sector
- 3.1.4 Organizing Tracks by Block
- 3.1.5 Nondata Overhead
- 3.1.6 The Cost of a Disk Access
- 3.1.7 Effect of Block Size on Performance: A UNIX Example
- 3.1.8 Disk as Bottleneck

3.2 Magnetic Tape
- 3.2.1 Organization of Data on Tapes
- 3.2.2 Estimating Tape Length Requirements
- 3.2.3 Estimating Data Transmission Times
- 3.2.4 Tape Applications

3.3 Disk versus Tape

3.4 Storage as a Hierarchy

3.5 A Journey of a Byte
- 3.5.1 The File Manager
- 3.5.2 The I/O Buffer
- 3.5.3 The Byte Leaves RAM: The I/O Processor and Disk Controller

3.6 Buffer Management
- 3.6.1 Buffer Bottlenecks
- 3.6.2 Buffering Strategies

3.7 I/O in UNIX
- 3.7.1 The Kernel
- 3.7.2 Linking File Names to Files
- 3.7.3 Normal Files, Special Files, and Sockets
- 3.7.4 Block I/O
- 3.7.5 Device Drivers
- 3.7.6 The Kernel and File Systems
- 3.7.7 Magnetic Tape and UNIX

Good design is always responsive to the constraints of the medium and to the environment. This is as true for file structure design as it is for designs in wood and stone. Given the ability to create, open, and close files, and to seek, read, and write, we can perform the fundamental operations of file *construction*. Now we need to look at the nature and limitations of the devices and systems used to store and retrieve files, preparing ourselves for file design.

If files were stored just in RAM, there would be no separate discipline called file structures. The general study of data structures would give us all the tools we would need to build file applications. But secondary storage devices are very different from RAM. One difference, as already noted, is that accesses to secondary storage take much more time than do accesses to RAM. An even more important difference, measured in terms of design impact, is that not all accesses are equal. Good file structure design uses knowledge of disk and tape performance to arrange data in ways that minimize access costs.

In this chapter we examine the characteristics of secondary storage devices, focusing on the constraints that shape our design work in the chapters that follow. We begin with a look at the major media used in the storage and processing of files, magnetic disks, and tapes. We follow this with an overview of the range of other devices and media used for secondary storage. Next, by following the journey of a byte, we take a brief look at the many pieces of hardware and software that become involved when a byte is sent by a program to a file on a disk. Finally, we take a closer look at one of the most important aspects of file management—buffering.

3.1 Disks

Compared to the time it takes to access an item in RAM, disk accesses are always expensive. However, not all disk accesses are *equally* expensive. The reason for this has to do with the way a disk drive works. Disk drives† belong to a class of devices known as *direct access storage devices* (DASDs) because they make it possible to access data *directly*. DASDs are contrasted with *serial devices*, the other major class of secondary storage devices. Serial devices use media such as magnetic tape that permit only serial access—a particular data item cannot be read or written until all of the data preceding it on the tape have been read or written in order.

Magnetic disks come in many forms. So-called *hard disks* offer high capacity and low cost per bit. Hard disks are the most common disk used in everyday file processing. *Floppy* disks are inexpensive, but they are slow and hold relatively little data. Floppies are good for backing up individual files or other floppies and for transporting small amounts of data. Removable disk packs are hard disks that can be mounted on the same drive at different times, providing a convenient form of backup storage that also makes it possible to access data directly.

Nonmagnetic disk media, especially optical discs, are becoming increasingly important for secondary storage. (See Appendix A for a full treatment of optical disc storage and its applications.)

3.1.1 The Organization of Disks

The information stored on a disk is stored on the surface of one or more platters (Fig. 3.1). The arrangement is such that the information is stored in successive *tracks* on the surface of the disk (Fig. 3.2). Each track is often

†When we use the terms *disks* or *disk drives*, we are referring to *magnetic* disk media.

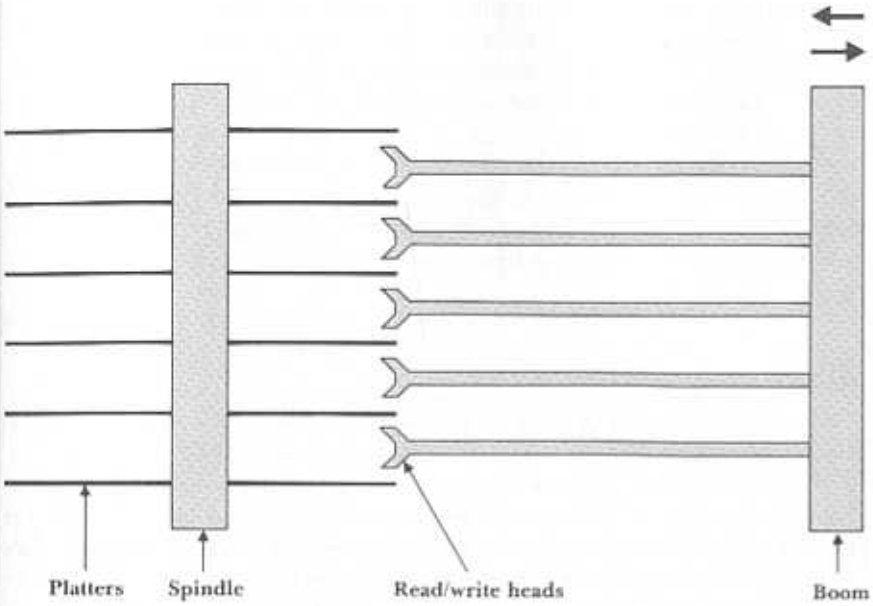


FIGURE 3.1 Schematic illustration of disk drive.

divided into a number of *sectors*. A sector is the smallest addressable portion of a disk. When a *READ( )* statement calls for a particular byte from a disk file, the computer operating system finds the correct surface, track, and sector, reads the entire sector into a special area in RAM called a *buffer*, and then finds the requested byte within that buffer.

If a disk drive uses a number of platters, it may be called a *disk pack*. The tracks that are directly above and below one another form a *cylinder* (Fig. 3.3). The significance of the cylinder is that all of the information on a single cylinder can be accessed without moving the arm that holds the read/write heads. Moving this arm is called *seeking*. This arm movement is usually the slowest part of reading information from a disk.

3.1.2 Estimating Capacities and Space Needs

Disks range in width from 2 to about 14 inches. They range in storage capacity from less than 400,000 bytes to billions of bytes. In a typical disk pack, the top and bottom platter each contribute one surface, and all other

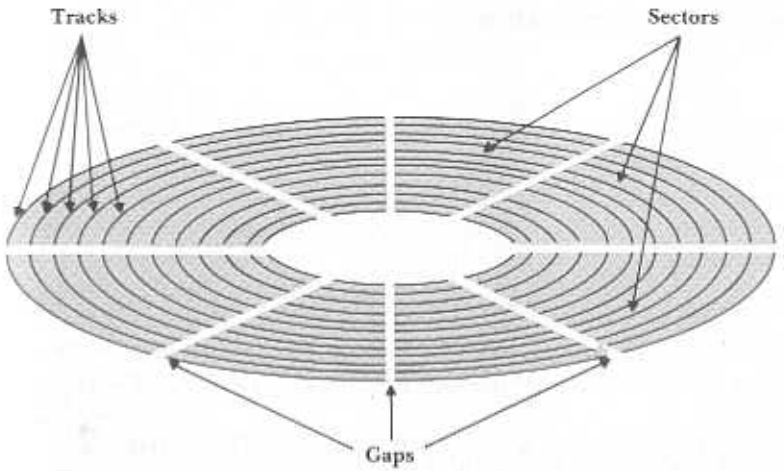
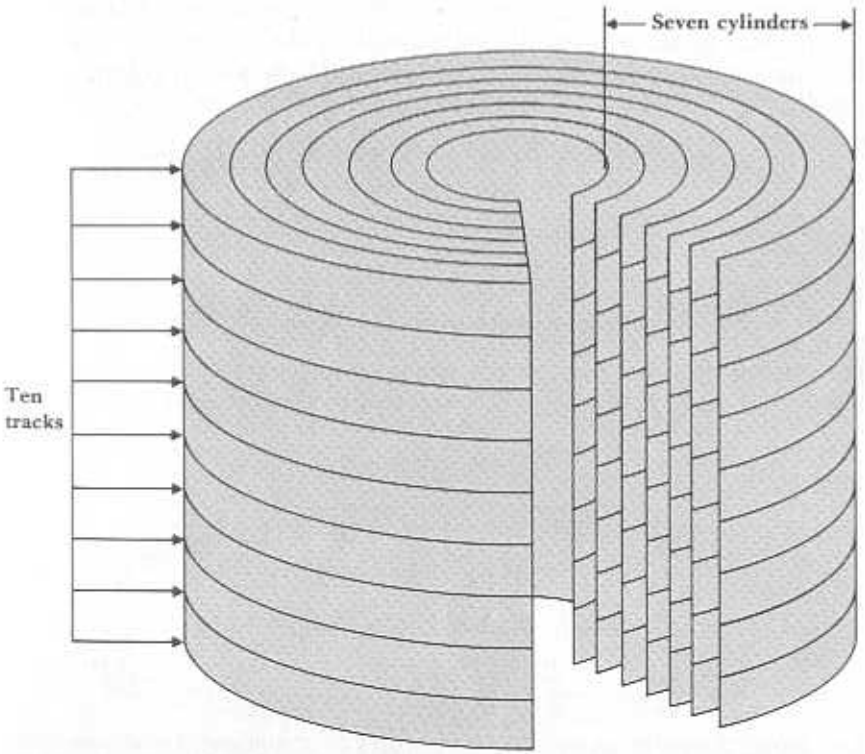


FIGURE 3.2 Surface of disk showing tracks and sectors.

FIGURE 3.3 Schematic illustration of disk drive viewed as a set of seven cylinders.



platters contribute two surfaces to the pack, so the number of tracks per cylinder is a function of the number of platters.

The amount of data that can be held on a track depends on how densely bits can be stored on the disk surface. (This in turn depends on the quality of the recording medium and the size of the read/write heads.) An inexpensive, low-density disk can hold about 4 kilobytes on a track, and 35 tracks on a surface. A top-of-the-line disk can hold about 50 kilobytes on a track, and more than 1,000 tracks on a surface. Table D.1 in Appendix D shows how a variety of disk drives compare in terms of capacity, performance, and cost.

Since a cylinder consists of a group of tracks, a track consists of a group of sectors, and a sector consists of a group of bytes, it is easy to compute track, cylinder, and drive capacities:

$$\text{Track capacity} = \text{number of sectors per track} \times \text{bytes per sector}$$

$$\text{Cylinder capacity} = \text{number of tracks per cylinder} \times \text{track capacity}$$

$$\text{Drive capacity} = \text{number of cylinders} \times \text{cylinder capacity}.$$

If we know the number of bytes in a file, we can use these relationships to compute the amount of disk space the file is likely to require. Suppose, for instance, that we want to store a file with 20,000 fixed-length data records on a "typical" 300-megabyte small computer disk with the following characteristics:

$$\text{Number of bytes per sector} = 512$$

$$\text{Number of sectors per track} = 40$$

$$\text{Number of tracks per cylinder} = 11$$

$$\text{Number of cylinders} = 1{,}331.$$

How many cylinders does the file require if each data record requires 256 bytes? Since each sector can hold two records, the file requires

$$\frac{20{,}000}{2} = 10{,}000 \text{ sectors}.$$

One cylinder can hold

$$40 \times 11 = 440 \text{ sectors}$$

so the number of cylinders required is approximately

$$\frac{10{,}000}{440} = 22.7 \text{ cylinders}.$$

Of course, it may be that a disk drive with 22.7 cylinders of available space does not have 22.7 *physically contiguous* cylinders available. In this likely case, the file might in fact have to be spread out over dozens, perhaps even hundreds, of cylinders.

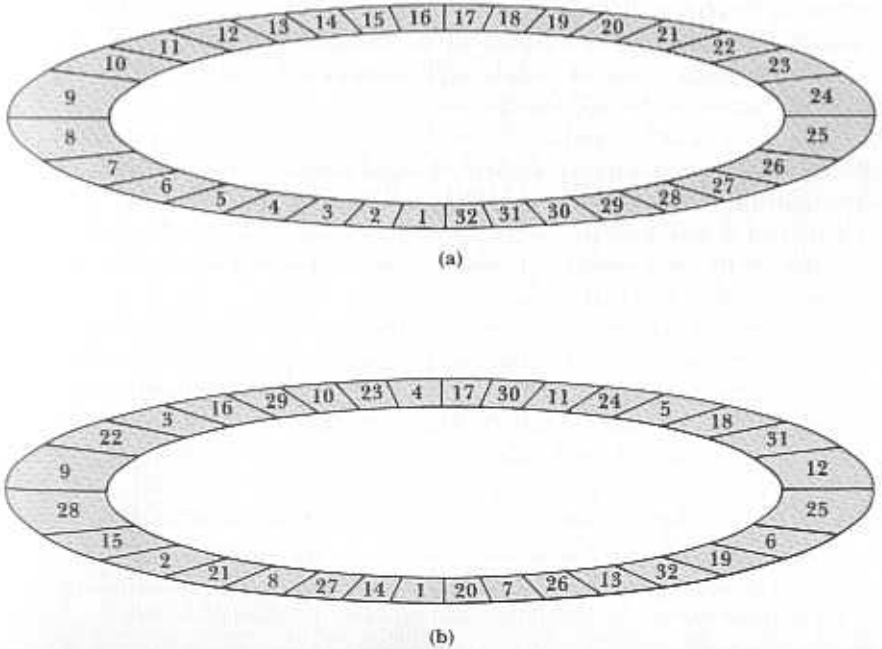
3.1.3 Organizing Tracks by Sector

There are two basic ways to organize data on a disk: by sector and by user-defined block. So far, we have only mentioned sector organizations. In this section we examine sector organizations more closely. In the following section we look at block organizations.

The Physical Placement of Sectors There are several views that one can have of the organization of sectors on a track. The simplest view, one that suffices for most users most of the time, is that sectors are adjacent, fixed-sized segments of a track that happen to hold a file (Fig. 3.4a). This is often a perfectly adequate way to view a file *logically*, but it may not be a good way to store sectors *physically*.

When you want to read a series of sectors that are all in the same track, one right after the other, you often cannot read *adjacent* sectors. That is

FIGURE 3.4 Two views of the organization of sectors on a 32-sector track.



because, after reading the data, it takes the disk controller a certain amount of time to process the received information before it is ready to accept more. So, if *logically* adjacent sectors were placed on the disk so they were also *physically* adjacent, we would miss the start of the following sector while we were processing the one we had just read in. Consequently, we would be able to read only one sector per revolution of the disk.

I/O system designers usually approach this problem by *interleaving* the sectors, leaving an interval of several physical sectors between logically adjacent sectors. Suppose our disk had an *interleaving factor* of 5. The assignment of logical sector content to the 32 physical sectors in a track is illustrated in Fig. 3.4(b). If you study this figure, you can see that it takes five revolutions to read the entire 32 sectors of a track. That is a big improvement over 32 revolutions.

Over the last year or two, controller speeds have improved so high-performance disks can now offer 1:1 interleaving. This means that successive sectors actually are physically adjacent, making it possible to read an entire track in a single revolution of the disk.

Clusters A third view of sector organization, also designed to improve performance, is the view maintained by that part of a computer's operating system that we call the *file manager*. When a program accesses a file, it is the file manager's job to map the logical parts of the file to their corresponding physical locations. It does this by viewing the file as a series of *clusters* of sectors. A cluster is a fixed number of contiguous sectors.† Once a given cluster has been found on a disk, all sectors in that cluster can be accessed without requiring an additional seek.

To view a file as a series of clusters and still maintain the sectored view, the file manager ties logical sectors to the physical clusters that they belong to by using a *file allocation table* (FAT). The FAT contains a list of all the clusters in a file, ordered according to the logical order of the sectors they contain. With each cluster entry in the FAT is an entry giving the physical location of the cluster (Fig. 3.5).

On many systems, the system administrator can decide how many sectors there should be in a cluster. For instance, in the standard physical disk structure used by VAX systems, the system administrator sets the cluster size to be used on a disk when the disk is initialized. The default value is three 512-byte sectors per cluster, but the cluster size may be set to any value between 1 and 65,535 sectors. Since clusters represent physically contiguous groups of sectors, larger clusters guarantee the ability to read

†It is not always *physically* contiguous; the degree of physical contiguity is determined by the interleaving factor.

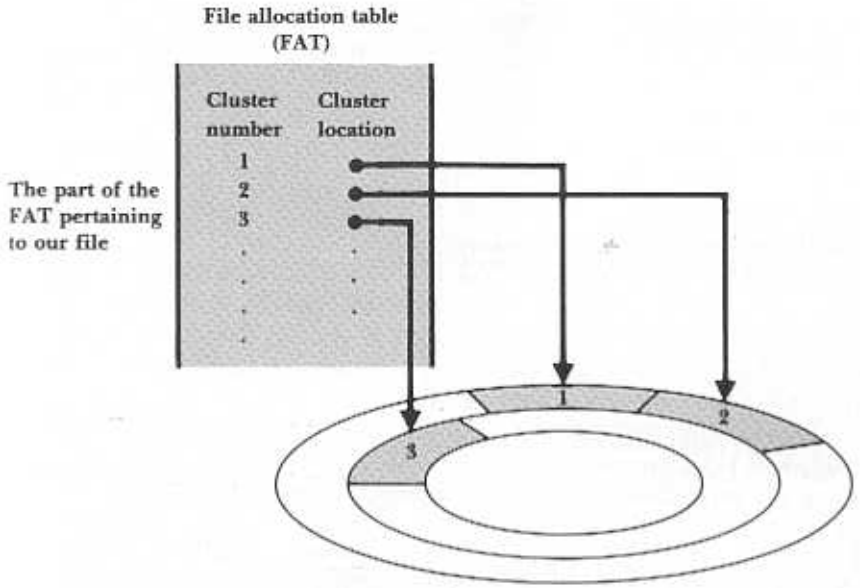


FIGURE 3.5 The file manager determines which cluster in the file has the sector that is to be accessed.

more sectors without seeking, so the use of large clusters can lead to substantial performance gains when a file is being processed sequentially.

Extents Our final view of sector organization represents a further attempt to emphasize physical contiguity of sectors in a file, hence minimizing seeking even more. (If you are getting the idea that the avoidance of seeking is an important part of file design, you are right.) If there is a lot of free room on a disk, it may be possible to make a file consist entirely of contiguous clusters. When this is the case, we say that the file consists of one *extent*: All of its sectors, tracks, and (if it is large enough) cylinders form one contiguous whole (Fig. 3.6a). This is a good situation, especially if the file is to be processed sequentially, because it means that the whole file can be accessed with a minimum amount of seeking.

If there is not enough contiguous space available to contain an entire file, the file is divided into two or more noncontiguous parts. Each part is an extent. When new clusters are added to a file, the file manager tries to make them physically contiguous to the previous end of the file, but if space is unavailable for this, it must add one or more extents (Fig. 3.6b). The

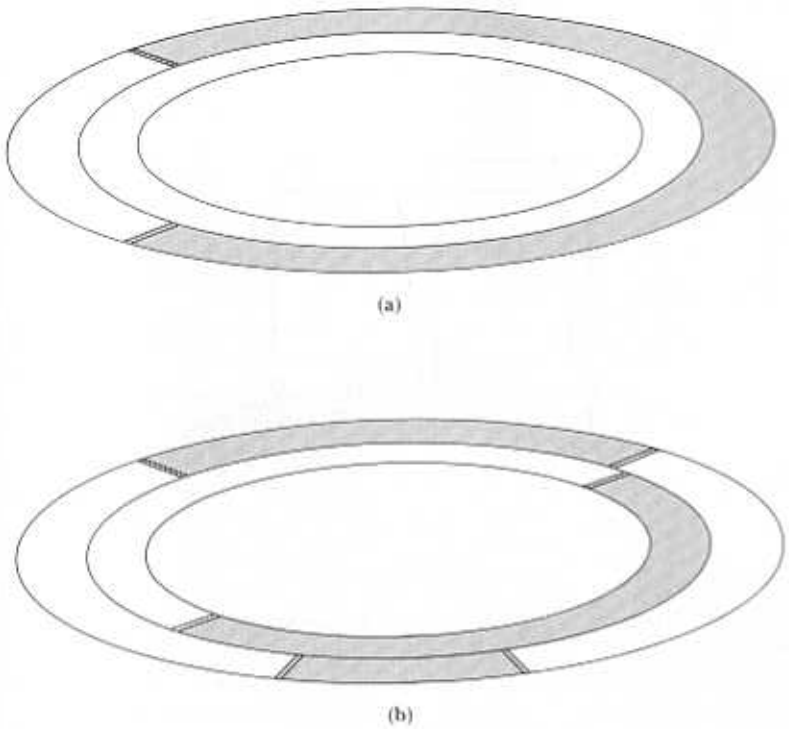


FIGURE 3.6 File extents (shaded area represents space on disk used by a single file).

most important thing to understand about extents is that as the number of extents in a file increases, the file becomes more spread out on the disk, and the amount of seeking required to process the file increases.

Fragmentation Generally, all sectors on a given drive must contain the same number of bytes. If, for example, the size of a sector is 512 bytes and the size of all records in a file is 300 bytes, there is no convenient fit between records and sectors. There are two ways to deal with this situation: Store only one record per sector, or allow records to *span* sectors, so the beginning of a record might be found in one sector and the end of it in another (Fig. 3.7).

The first option has the advantage that any record can be retrieved by retrieving just one sector, but it has the disadvantage that it might leave an enormous amount of unused space within each sector. This loss of space within a sector is called *internal fragmentation*. The second option has the

advantage that it loses no space from internal fragmentation, but it has the disadvantage that some records may be retrieved only by accessing two sectors.

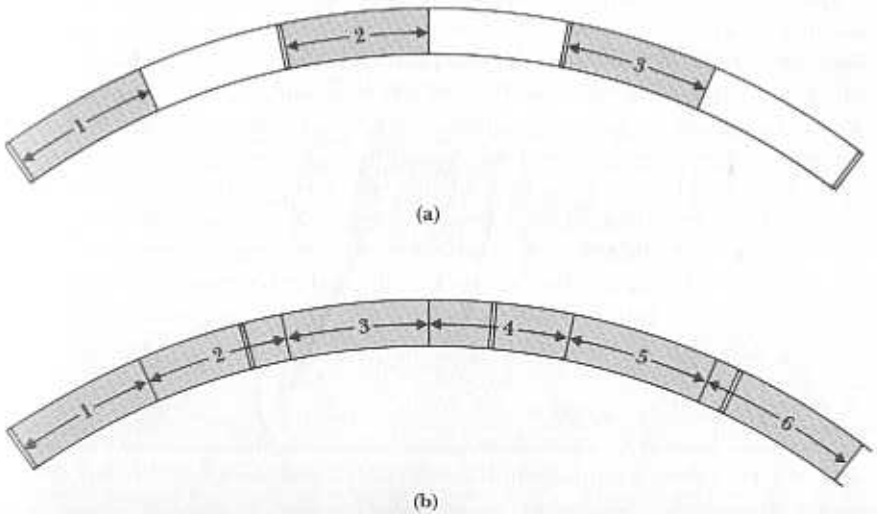
Another potential source of internal fragmentation results from the use of clusters. Recall that a cluster is the smallest unit of space that can be allocated for a file. When the number of bytes in a file is not an exact multiple of the cluster size, there will be internal fragmentation in the last extent of the file. For instance, if a cluster consists of three 512-byte sectors, a file containing one byte would use up 1,536 bytes on the disk; 1,535 bytes would be wasted due to internal fragmentation.

Clearly, there are important trade-offs in the use of large cluster sizes. A disk that is expected to have mainly large files that will often be processed sequentially would usually be given a large cluster size, since internal fragmentation would not be a big problem and the performance gains might be great. A disk holding smaller files or files that are usually accessed only randomly would normally be set up with small clusters.

3.1.4 Organizing Tracks by Block

Sometimes disk tracks are *not* divided into sectors, but into integral numbers of user-defined *blocks* whose size can vary. (*Note:* The word *block*

FIGURE 3.7 Alternate record organization within sectors (shaded areas represent data records, and unshaded areas represent unused space).



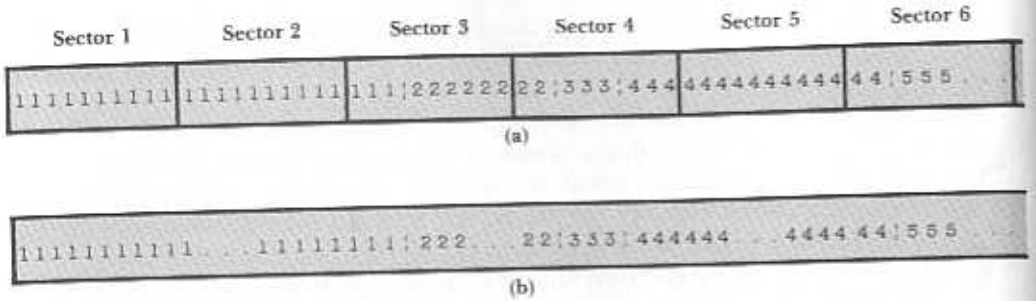


FIGURE 3.8 Sector organization versus block organization.

has a different meaning in the context of the UNIX I/O system. See section 3.7 for details.) When the data on a track is organized by block, this usually means that the amount of data transferred in a single I/O operation can vary depending on the needs of the software designer, not the hardware. Blocks can normally be either fixed or variable in length, depending on the requirements of the file designer. As with sectors, blocks are often referred to as physical records. (Sometimes the word *block* is used as a synonym for a sector or group of sectors. To avoid confusion, we do not use it in that way here.) Figure 3.8 illustrates the difference between one view of data on a sectored track and that of a blocked track.

A *block* organization does not present the sector-spanning and fragmentation problems of sectors because blocks can vary in size to fit the logical organization of the data. A block is usually organized to hold an integral number of logical records. The term *blocking factor* is used to indicate the number of records that are to be stored in each block in a file. Hence, if we had a file with 300-byte records, a block-addressing scheme would let us define a block to be some convenient multiple of 300 bytes, depending on the needs of the program. No space would be lost to internal fragmentation, and there would be no need to load two blocks to retrieve one record.

Generally speaking, blocks are superior to sectors when it is desirable to have the physical allocation of space for records correspond to their logical organization. (There are disk drives that allow both sector-addressing and block-addressing, but we do not describe them here. See Bohl, 1981.)

In block-addressing schemes, each block of data is usually accompanied by one or more *subblocks* containing extra information about the data block. Typically there is a *count subblock* that contains (among other things) the number of bytes in the accompanying data block (Fig. 3.9a). There may also be a *key subblock* containing the key for the last record in the data block

(Fig. 3.9b). When *key* subblocks are used, a track can be searched by the disk controller for a block or record identified by a given key. This means that a program can ask its disk drive to search among all the blocks on a track for a block with a desired key. This approach can result in much more efficient searches than are normally possible with sector-addressable schemes, in which keys cannot generally be interpreted without first loading them into primary memory.

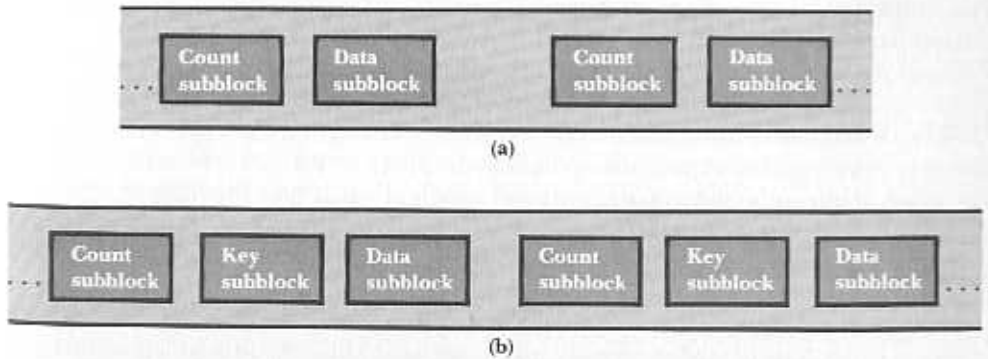
3.1.5 Nondata Overhead

Both blocks and sectors require that a certain amount of space be taken up on the disk in the form of *nondata overhead*. Some of the overhead consists of information that is stored on the disk during *preformatting*, which is done before the disk can be used.

On sector-addressable disks, preformatting involves storing, at the beginning of each sector, such information as sector address, track address, and condition (whether the sector is usable or defective). Preformatting also involves placing gaps and synchronization marks between fields of information to help the read/write mechanism distinguish between them. This nondata overhead usually is of no concern to the programmer. When the sector size is given for a certain drive, the programmer can assume that this is the amount of actual data that can be stored in a sector.

On a block-organized disk, some of the nondata overhead is invisible to the programmer, but some of it must be accounted for by the programmer. Since subblocks and interblock gaps have to be provided with every block,

FIGURE 3.9 Block addressing requires that each physical data block be accompanied by one or more subblocks containing information about its contents.



there is generally more nondata information provided with blocks than with sectors. Also, since the number and sizes of blocks can vary from one application to another, the relative amount of space taken up by overhead can vary when block addressing is used. This is illustrated in the following example.

Suppose we have a block-addressable disk drive with 20,000 bytes per track, and the amount of space taken up by subblocks and interblock gaps is equivalent to 300 bytes per block. We want to store a file containing 100-byte records on the disk. How many records can be stored per track if the blocking factor is 10, or if it is 60?

1. If there are 10 100-byte records per block, each block holds 1,000 bytes of data and uses 300 + 1,000, or 1,300, bytes of track space when overhead is taken into account. The number of blocks which can fit on a 20,000-byte track can be expressed as

$$\left\lfloor \frac{20{,}000}{1{,}300} \right\rfloor = \lfloor 15.38 \rfloor = 15.$$

   So 15 blocks, or 150 records, can be stored per track. (Note that we have to take the *floor* of the result because a block cannot span two tracks.)
2. If there are 60 100-byte records per block, each block holds 6,000 bytes of data and uses 6,300 bytes of track space. The number of blocks per track can be expressed as

$$\left\lfloor \frac{20{,}000}{6{,}300} \right\rfloor = 3.$$

   So 3 blocks, or 180 records, can be stored per track.

Clearly, the larger blocking factor can lead to more efficient use of storage. When blocks are larger, fewer blocks are required to hold a file, so there is less space consumed by the 300 bytes of overhead that accompany each block.

Can we conclude from this example that larger blocking factors always lead to more efficient storage utilization? Not necessarily. Since we can put only an integral number of blocks on a track, and since tracks are fixed in length, we almost always lose some space at the end of a track. Here we have the internal fragmentation problem again, but this time it applies to fragmentation within a *track*. The greater the block size, the greater potential amount of internal track fragmentation. What would have happened if we had chosen a blocking factor of 98 in the preceding example? What about 97?

The flexibility introduced by the use of blocks, rather than sectors, can result in savings in time and efficiency, since it lets the programmer determine to a large extent how data are to be organized physically on a disk. On the negative side, blocking schemes *require* the programmer and/or operating system to do the extra work of determining the data organization. Also, the very flexibility introduced by the use of blocking schemes precludes the synchronization of I/O operations with the physical movement of the disk, which sectoring permits. This means that strategies such as sector interleaving cannot be used to improve performance.

3.1.6 The Cost of a Disk Access

To give you a feel for the factors contributing to the total amount of time needed to access a file on a fixed disk, we calculate some access times. A disk access can be divided into three distinct physical operations, each with its own cost: *seek time*, *rotational delay*, and *transfer time*.

Seek Time Seek time is the time required to move the access arm to the correct cylinder. The amount of time spent seeking during a disk access depends, of course, on how far the arm has to move. If we are accessing a file sequentially and the file is packed into several consecutive cylinders, seeking needs to be done only after all of the tracks on a cylinder have been processed, and even then the read/write head needs to move the width of only one track. At the other extreme, if we are alternately accessing sectors from two files that are stored at opposite extremes on a disk (one at the innermost cylinder, one at the outermost cylinder), seeking is very expensive.

Seeking is likely to be more costly in a multiuser environment, where several processes are contending for use of the disk at one time, than in a single-user environment, where disk usage is dedicated to one process.

Since seeking can be very costly, system designers often go to great extremes to minimize seeking. In an application that merges three files, for example, it is not unusual to see the three input files stored on three different drives and the output file stored on a fourth drive, so no seeking need be done as I/O operations jump from file to file.

Since it is usually impossible to know exactly how many tracks will be traversed in every seek, we usually try to determine the *average seek time* required for a particular file operation. If the starting and ending positions for each access are random, it turns out that the average seek traverses one third of the total number of cylinders that the read/write head ranges over.[†] Manufacturers' specifications for disk drives often list this figure as the

[†]Derivations of this result, as well as more detailed and refined models, can be found in Wiederhold (1983), Knuth (1973b), Teory and Fry (1982), and Salzberg (1988).

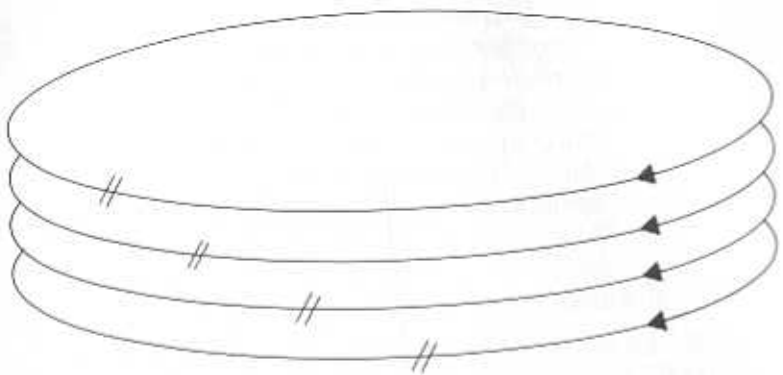


FIGURE 3.10 When a single file can span several tracks on a cylinder, we can stagger the beginnings of the tracks to avoid rotational delay when moving from track to track during sequential access.

average seek time for the drives. Most hard disks available today (1991) have average seek times of less than 40 milliseconds (msec), and high-performance disks have average seek times as low as 10 msec.

Rotational Delay Rotational delay refers to the time it takes for the disk to rotate so the sector we want is under the read/write head. Hard disks usually rotate at about 3,600 rpm, which is one revolution per 16.7 msec. On average, the rotational delay is half a revolution, or about 8.3 msec. On floppy disks, which often rotate at only 360 rpm, average rotational delay is a sluggish 83.3 msec.

As in the case of seeking, these averages apply only when the read/write head moves from some random place on the disk surface to the target track. In many circumstances, rotational delay can be much less than the average. For example, suppose that you have a file that requires two or more tracks, that there are plenty of available tracks on one cylinder, and that you write the file to disk sequentially, with one write call. When the first track is filled, the disk can immediately begin writing to the second track, without any rotational delay. The "beginning" of the second track is effectively staggered by just the amount of time it takes to switch from the read/write head on the first track to the read/write head on the second. Rotational delay, as it were, is virtually nonexistent. Furthermore, when you read the file back, the position of data on the second track ensures that there is no rotational delay in switching from one track to another. Figure 3.10 illustrates this staggered arrangement.

Transfer Time Once the data we want is under the read/write head, it can be transferred. The transfer time is given by the formula

$$\text{Transfer time} = \frac{\text{number of bytes transferred}}{\text{number of bytes on a track}} \times \text{rotation time.}$$

If a drive is sectored, the transfer time for one sector depends on the number of sectors on a track. For example, if there are 32 sectors per track, the time required to transfer one sector would be 1/32nd of a revolution, or 0.5 msec.

Some Timing Computations Let's look at two different file processing situations that show how different types of file access can affect access times. We will compare the time it takes to access a file *in sequence* with the time it takes to access all of the records in the file *randomly*. In the former case, we use as much of the file as we can whenever we access it. In the random-access case, we are able to use only one record on each access.

The basis for our calculations is a "typical" 300-megabyte fixed disk described in Table 3.1. This particular disk is typical of one that might be used with a workstation in 1991. Although it is typical only of a certain class of fixed disk, the observations we draw as we perform these calculations are quite general. The disks used with larger, more expensive computers are bigger and faster than this disk, but the nature and relative costs of the factors contributing to total access times are essentially the same.

TABLE 3.1 Specifications of disk drive used in examples in text

Minimum (track-to-track) seek time	6 msec
Average seek time	18 msec
Rotational delay	8.3 msec
Maximum transfer rate ( )	16.7 msec/track, or 1,229 bytes/msec
Bytes per sector	512
Sectors per track	40
Tracks per cylinder	11
Tracks per surface	1,331
Interleave factor	1
Cluster size	8 sectors
Smallest extent size	5 clusters

Since our drive uses a cluster size of 8 sectors (4,096 bytes) and the smallest extent is 5 clusters, space is allocated for storing files in one-track units. Sectors are interleaved with an interleave factor of 1, so data on a given track can be transferred at the stated transfer rate.

Let's suppose that we wish to know how long it will take, using this drive, to read a 2,048-K-byte file that is divided into 8,000 256-byte records. First we need to know how the file is distributed on the disk. Since the 4,096-byte cluster holds 16 records, the file will be stored as a sequence of 500 4,096-byte clusters. Since the smallest extent size is 5 clusters, the 500 clusters are stored as 100 extents, occupying 100 tracks.

This means that the disk needs 100 tracks to hold the entire 2,048 K bytes that we want to read. We assume a situation in which the 100 tracks are randomly dispersed over the surface of the disk. (This is an extreme situation chosen to dramatize the point we want to make. Still, it is not so extreme that it could not easily occur on a typical overloaded disk that has a large number of small files.)

Now we are ready to calculate the time it would take to read the 2,048-K-byte file from the disk. We first estimate the time it takes to read the file sector by sector *in sequence*. This process involves the following operations for each track:

Average seek	18	msec
Rotational delay	8.3	msec
Read one track	16.7	msec
Total	43	msec.

We want to find and read 100 tracks, so the

$$\text{Total time} = 100 \times 43 \text{ msec} = 4{,}300 \text{ msec} = 4.3 \text{ seconds.}$$

Now let's calculate the time it would take to read in the same 8,000 records using *random access* rather than sequential access. In other words, rather than being able to read one sector right after another, we assume that we have to access the records in some order that requires jumping from track to track every time we read a new sector. This process involves the following operations for each record:

Average seek	18	msec
Rotational delay	8.3	msec
Read one cluster $\left(\frac{1}{5} \times 16.7\right)$	3.3	msec
Total	29.6	msec

$$\text{Total time} = 8{,}000 \times 29.6 \text{ msec} = 236{,}800 \text{ msec} = 236.8 \text{ seconds.}$$

This difference in performance between sequential access and random access is very important. If we can get to the right location on the disk and read a lot of information sequentially, we are clearly much better off than we are if we have to jump around, *seeking* every time we need a new record. Remember that seek time is very expensive; when we are performing disk operations we should try to minimize seeking.

3.1.7 Effect of Block Size on Performance: A UNIX Example

In deciding how best to organize disk storage allocation for several versions of BSD UNIX, the Computer Systems Research Group (CSRG) in Berkeley investigated the trade-offs between block size and performance in a UNIX environment (Leffler et al., 1989). The results of their research provide an interesting case study involving trade-offs between block size, fragmentation, and access time.

The CSRG research indicated that minimum block size of 512 bytes, standard at the time on UNIX systems, was not very efficient in a typical UNIX environment. Files that were several blocks long often were scattered over many cylinders, resulting in frequent seeks and thereby significantly decreasing throughput. The researchers found that doubling the block size to 1,024 bytes improved performance by more than a factor of 2. But even with 1,024-byte blocks, they found that throughput was only about 4% of the theoretical maximum. Eventually, they found that 4,096-byte blocks provided the fastest throughput, but this led to large amounts of wasted space due to internal fragmentation. These results are summarized in Table 3.2.

TABLE 3.2 The amount of wasted space as a function of block size

Space Used (Mbyte)	Percent Waste	Organization
775.2	0.0	Data only, no separation between files
807.8	4.2	Data only, each file starts on 512-byte boundary
828.7	6.9	Data + inodes, 512-byte block UNIX file system
866.5	11.8	Data + inodes, 1,024-byte block UNIX file system
948.5	22.4	Data + inodes, 2,048-byte block UNIX file system
1,128.3	45.6	Data + inodes, 4,096-byte block UNIX file system

From *The Design and Implementation of the 4.3BSD UNIX Operating System*, Leffler et al., p. 198.

To gain the advantages of both the 4,096-byte and the 512-byte systems, the Berkeley group implemented a variation of the cluster concept (see section 3.1.3). In the new implementation, they allocate 4,096-byte blocks for files that are big enough to need them; but for smaller files, they allow the large blocks to be divided into one or more fragments. With a fragment size of 512 bytes, as many as eight small files can be stored in one block, greatly reducing internal fragmentation. With the 4,096/512 system, wasted space was found to decline to about 12%.

3.1.8 Disk as Bottleneck

Disk performance is increasing steadily, even dramatically, but disk speeds still lag far behind local network speeds. A high-performance disk drive with 50 K bytes per track can transmit at a peak rate of about 3 megabytes per second, and only a fraction of that under normal conditions. High-performance networks, in contrast, can transmit at rates of as much as 100 megabytes per second. The result can often mean that a process is *disk bound*—the network and the CPU have to wait inordinate lengths of time for the disk to transmit data.

A number of techniques are used to solve this problem. One is multiprogramming, in which the CPU works on other jobs while waiting for the data to arrive. But if multiprogramming is not available, or if the process simply cannot afford to lose so much time waiting for the disk, ways must be found to speed up disk I/O.

One technique that is now offered on many high-performance systems is called *striping*. Disk striping involves splitting the parts of a file on several different drives, then letting the separate drives deliver parts of the file to the network simultaneously.

For example, suppose we have a 10-megabyte file spread across 20 high-performance (3 megabytes per second) drives that hold 50 K per track. The first drive has the first 50 K of the file, the second drive has the second 50 K, etc., through the twentieth drive. The first drive also holds the twenty-first 50 K, and so forth until 10 megabytes are stored. Collectively, the 20 drives can deliver to the network 250 K per revolution, a combined rate of 60 megabytes per second.

Disk striping exemplifies an important concept that we see more and more in system configurations—*parallelism*. Whenever there is a bottleneck at some point in the system, consider duplicating the thing that is the source of the bottleneck, and configure the system so several of them operate in parallel.

Another approach to solving the disk bottleneck is to avoid accessing the disk at all. As the cost of RAM steadily decreases, more and more users are using RAM to hold data that a few years ago had to be kept on a disk. Two effective ways in which RAM can be used to replace secondary storage are RAM disks and disk caches.

A *RAM disk* is a large part of RAM configured to simulate the behavior of a mechanical disk in every respect except speed and volatility. Since data can be located in RAM without a seek or rotational delay, RAM disks can provide much faster access than mechanical disks. Since RAM is normally volatile, the contents of a RAM disk are lost when the computer is turned off. RAM disks are often used in place of floppy disks because they are much faster than floppies and because relatively little RAM is needed to simulate a typical floppy disk.

A *disk cache*[†] is a large block of RAM configured to contain *pages* of data from a disk. A typical disk-caching scheme might use a 256-K cache with a disk. When data is requested from secondary memory, the file manager first looks into the disk cache to see if it contains the page with the requested data. If it does, the data can be processed immediately. Otherwise, the file manager reads the page containing the data from disk, replacing some page already in the disk cache.

Cache memory can provide substantial improvements in performance, especially when a program's data access patterns exhibit a high degree of *locality*. Locality exists in a file when blocks that are accessed in close temporal sequence are stored close to one another on the disk. When a disk cache is used, blocks that are close to one another on the disk are much more likely to belong to the page or pages that are read in with a single read, diminishing the likelihood that extra reads are needed for extra accesses.

RAM disks and cache memory are examples of *buffering*, a very important and frequently used family of I/O techniques. We take a closer look at buffering in section 3.6.

In these three techniques we see once again examples of the need to make trade-offs in file processing. With RAM disks and disk caches, there is tension between the cost/capacity advantages of disk over RAM, on the one hand, and the speed of RAM on the other. Striping provides opportunities to increase throughput enormously, but at the cost of a more complex and sophisticated disk management system. Good file design balances these tensions and costs creatively.

[†]The term *cache* (as opposed to *disk cache*) generally refers to a very high-speed block of primary memory that performs the same types of performance-enhancing operations with respect to RAM that a disk cache does with respect to secondary memory.

3.2 Magnetic Tape

Magnetic tape units belong to a class of devices that provide no direct accessing facility but that can provide very rapid sequential access to data. Tapes are compact, stand up well under different environmental conditions, are easy to store and transport, and are less expensive than disks.

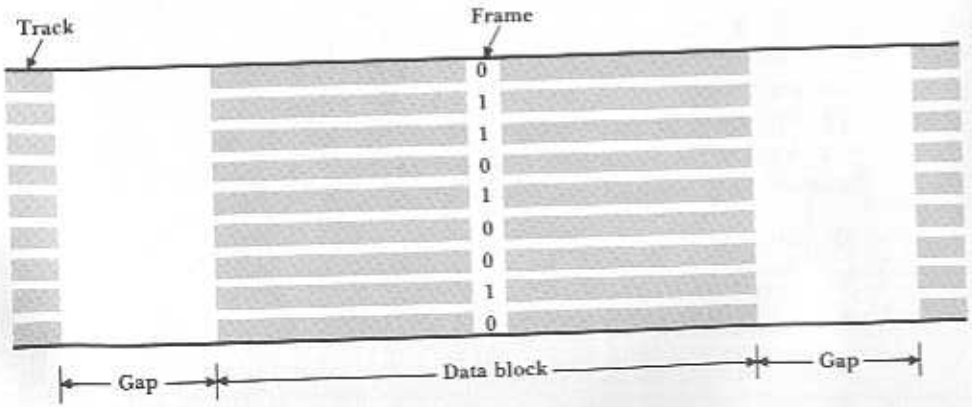
3.2.1 Organization of Data on Tapes

Since tapes are accessed sequentially, there is no need for addresses to identify the locations of data on a tape. On a tape, the logical position of a byte within a file corresponds directly to its physical position relative to the start of the file. We may envision the surface of a typical tape as a set of parallel tracks, each of which is a sequence of bits. If there are nine tracks (see Fig. 3.11), the nine bits that are at corresponding positions in the nine respective tracks are taken to constitute one byte, plus a *parity bit*. So a byte can be thought of as a one-bit-wide slice of tape. Such a slice is called a *frame*.

The parity bit is not part of the data but is used to check the validity of the data. If *odd parity* is in effect, this bit is set to make the number of 1 bits in the frame *odd*. Even parity works similarly but is rarely used with tapes.

Frames (bytes) are grouped into data blocks whose size can vary from a few bytes to many kilobytes, depending on the needs of the user. Since tapes are often read one block at a time, and since tapes cannot stop or start instantaneously, blocks are separated by *interblock gaps*, which contain no

FIGURE 3.11 Nine-track tape.



information and are long enough to permit stopping and starting. When tapes use odd parity, no valid frame can contain all 0 bits, so a large number of consecutive 0 frames is used to fill the interrecord gap.

Tape drives come in many shapes, sizes, and speeds. Performance differences among drives can usually be measured in terms of three quantities:

- Tape density—commonly 800, 1,600, or 6,250 bits per inch (bpi) per track, but recently as much as 30,000 bpi;
- Tape speed—commonly 30 to 200 inches per second (ips); and
- Size of interblock gap—commonly between 0.3 inch and 0.75 inch.

Note that a 6,250-bpi nine-track tape contains 6,250 bits per inch per track, and 6,250 *bytes* per inch when the full nine tracks are taken together. Thus, in the computations that follow, 6,250 bpi is usually taken to mean 6,250 bytes of data per inch.

3.2.2 Estimating Tape Length Requirements

Suppose we want to store a backup copy of a large mailing list file with one million 100-byte records. If we want to store the file on a 6,250-bpi tape that has an interblock gap of 0.3 inches, how much tape is needed?

To answer this question we first need to determine what takes up space on the tape. There are two primary contributors: interblock gaps and data blocks. For every data block there is an interblock gap. If we let

b = the physical length of a data block,

g = the length of an interblock gap, and

n = the number of data blocks,

then the space requirement s for storing the file is

$$s = n \times (b + g).$$

We know that g is 0.3 inch, but we do not know what b and n are. In fact, b is whatever we want it to be, and n depends on our choice of b. Suppose we choose each data block to contain one 100-byte record. Then b, the length of each block, is given by

$$b = \frac{\text{block size (bytes per block)}}{\text{tape density (bytes per inch)}} = \frac{100}{6{,}250} = 0.016 \text{ inch},$$

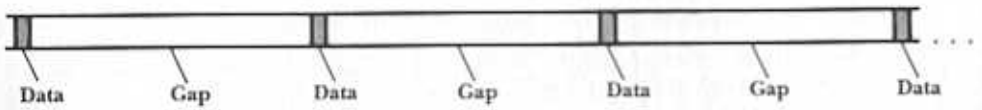
and n, the number of blocks, is one million (one per record).

The number of records stored in a physical block is called the *blocking factor*. It has the same meaning that it had when it was applied to the use of

blocks for disk storage. The blocking factor we have chosen here is 1 because each block has only one record. Hence, the space requirement for the file is

$$\begin{aligned} s &= 1{,}000{,}000 \times (0.016 + 0.3) \text{ inch} \\ &= 1{,}000{,}000 \times 0.316 \text{ inch} \\ &= 316{,}000 \text{ inches} \\ &= 26{,}333 \text{ feet.} \end{aligned}$$

Magnetic tapes range in length from 300 feet to 3,600 feet, with 2,400 feet being the most common length. Clearly, we need quite a few 2,400-foot tapes to store the file. Or do we? You may have noticed that our choice of block size was not a very smart one from the standpoint of space utilization. The interblock gaps in the physical representation of the file take up about *19 times* as much space as the data blocks do. If we were to take a snapshot of our tape, it would look something like this:

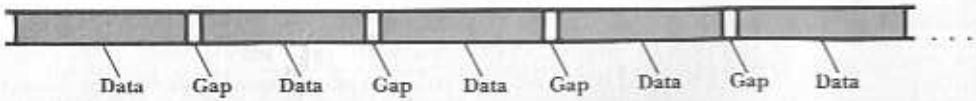


Most of the space on the tape is not used!

Clearly, we should consider increasing the relative amount of space used for actual data if we want to try to squeeze the file onto one 2,400-foot tape. If we increase the blocking factor, we can *decrease* the number of blocks, which decreases the number of interblock gaps, which in turn decreases the amount of space consumed by interblock gaps. For example, if we increase the blocking factor from 1 to 50, the number of blocks becomes

$$n = \frac{1{,}000{,}000}{50} = 20{,}000,$$

and the space requirement for interblock gaps decreases from 300,000 inches to 6,000 inches. The space requirement for the data is of course the same as it was previously. What has changed is the *relative* amount of space occupied by the gaps, as compared to the data. Now a snapshot of the tape would look much different:



We leave it to you to show that the file can fit easily on one 2,400-foot tape when a blocking factor of 50 is used.

When we compute the space requirements for our file, we produce numbers that are quite specific to our file. A more general measure of the effect of choosing different block sizes is *effective recording density*. The effective recording density is supposed to reflect the amount of actual data that can be stored per inch of tape. Since this depends exclusively on the relative sizes of the interblock gap and the data block, it can be defined as

$$\frac{\text{number of bytes per block}}{\text{number of inches required to store a block}}$$

When a blocking factor of 1 is used in our example, the number of bytes per block is 100, and the number of inches required to store a block is 0.316. Hence, the effective recording density is

$$\frac{100 \text{ bytes}}{0.316 \text{ inches}} = 316.4 \text{ bpi},$$

which is a far cry from the *nominal* recording density of 6,250 bpi.

Either way you look at it, space utilization is sensitive to the relative sizes of data blocks and interblock gaps. Let us now see how they affect the amount of *time* it takes to transmit tape data.

3.2.3 Estimating Data Transmission Times

If you understand the role of interblock gaps and data block sizes in determining effective recording density, you can probably see immediately that these two factors also affect the rate of data transmission. Two other factors that affect the rate of data transmission to or from tape are the nominal recording density and the speed with which the tape passes the read/write head. If we know these two values, we can compute the *nominal data transmission rate:*

$$\text{Nominal rate} = \text{tape density (bpi)} \times \text{tape speed (ips)}.$$

Hence, our 6,250-bpi, 200-ips tape has a nominal transmission rate of

$$\begin{aligned} 6{,}250 \times 200 &= 1{,}250{,}000 \text{ bytes/sec} \\ &= 1{,}250 \text{ kilobytes/sec.} \end{aligned}$$

This rate is competitive with most disk drives.

But what about those interblock gaps? Once our data gets dispersed by interblock gaps, the *effective transmission rate* certainly suffers. Suppose, for example, that we use our blocking factor of 1 with the same file and tape

drive discussed in the preceding section (1,000,000 100-byte records, 0.3-inch gap). We saw that the effective recording density for this tape organization is 316.4 bpi. If the tape is moving at a rate of 200 ips, then its effective transmission rate is

$$316.4 \times 200 = 63{,}280 \text{ bytes/sec}$$
$$= 63.3 \text{ kilobytes/sec},$$

a rate that is about *one twentieth* of the nominal rate!

It should be clear that a blocking factor larger than 1 improves on this result, and that a substantially larger blocking factor improves on it substantially.

Although there are other factors that can influence performance, block size is generally considered to be the one variable with the greatest influence on space utilization and data transmission rate. The other factors we have included—gap size, tape speed, and recording density—are often beyond the control of the user. Another factor that can sometimes be important is the time it takes to start and stop the tape. We consider start/stop time in the exercises at the end of this chapter.

3.2.4 Tape Applications

Magnetic tape is an appropriate medium for sequential processing applications *if* the files being processed are not likely also to be used in applications that require direct access. For example, consider the problem of updating a mailing list for a monthly periodical. Is it essential that the list be kept absolutely current, or is a monthly update of the list sufficient?

If information must be up-to-the-minute, then the medium must permit direct access so individual updates can be made immediately. But if the mailing list needs to be current only when mailing labels are printed, all of the changes that occur during the course of a month can be collected in one batch and put into a transaction file that is sorted in the same way that the mailing list is sorted. Then a program that reads through the two files simultaneously can be executed, making all the required changes in one pass through the data.

Since tape is relatively inexpensive, it is an excellent medium for storing data offline. At current prices, a removable disk pack that holds 150 megabytes costs about 30 times as much as a reel of tape that, properly blocked, can hold the same amount. Tape is a good medium for archival storage and for transporting data, as long as the data does not have to be available on short notice for direct processing.

A special kind of tape drive, a *streaming tape drive*, is used widely for nonstop, high-speed dumping of data to and from disks. Generally less expensive than general-purpose tape drives, it is also less suited for processing that involves much starting and stopping.

3.3 Disk versus Tape

In the past, magnetic tape and magnetic disk accounted for the lion's share of all secondary storage applications. Disk was excellent for random access and storage of files for which immediate access was desired; tape was ideal for processing data sequentially and for long-term storage of files. Over time, these roles have changed somewhat in favor of disk.

The major reason that tape was preferable to disk for sequential processing is that tapes are dedicated to one process, while disk generally serves several processes. This means that between accesses a disk read/write head tends to move away from the location where the next sequential access will occur, resulting in an expensive seek; while the tape drive, being dedicated to one process, pays no such price in seek time.

This problem of excessive seeking has gradually diminished, and disk has taken over much of the secondary storage niche previously occupied by tape. This change is largely due to the continued dramatic decreases in the cost of disk and RAM storage. To fully understand this change, we need to understand the role of RAM buffer space in performing I/O.† Briefly, it is that performance depends largely on how big a chunk of a file we can transmit at any time; as more RAM space becomes available for I/O buffers, the number of accesses decreases correspondingly, which means that the number of seeks required goes down as well. Most systems now available, even small systems, have enough RAM available to decrease the number of accesses required to process most files to a level that makes disk quite competitive with tape for sequential processing. This change, added to the superior versatility and decreasing costs of disks, has resulted in use of disk for most sequential processing, which in the past was primarily the domain of tape.

This is not to say that tapes should not be used for sequential processing. If a file is kept on tape, and there are enough drives available to use them for sequential processing, it may be more efficient to process the file directly from tape than to stream it to disk and then process it sequentially.

Although it has lost ground to disk in sequential processing applications, tape remains important as a medium for long-term archival storage. Tape is still far less expensive than magnetic disk, and it is very easy and fast

†Techniques for RAM buffering are covered in section 3.6.

to stream large files or sets of files between tape and disk. In this context, tape has emerged as one of our most important media (along with CD-ROM) for *tertiary* storage.

3.4 Storage as a Hierarchy

Although the best mixture of devices for a computing system depends on the needs of the system's users, we can imagine any computing system as a hierarchy of storage devices of different speed, capacity, and cost. Figure 3.12 summarizes the different types of storage found at different levels in such hierarchies and shows approximately how they compare in terms of access time, capacity, and cost.

FIGURE 3.12 Approximate comparisons of types of storage, circa 1991.

Types of memory	Devices and media	Access times (sec)	Capacities (bytes)	Cost (cents/bit)
Primary				
Registers	Core and semiconductors	10^{-9}–10^{-5}	10^{0}–10^{9}	10^{0}–10^{-3}
RAM				
RAM disk and disk cache				
Secondary				
Direct-access	Magnetic disks	10^{-3}–10^{-1}	10^{8}–10^{9}	10^{-2}–10^{-5}
Serial	Tape and mass storage	10^{1}–10^{2}	10^{9}–10^{11}	10^{-5}–10^{-7}
Offline				
Archival and backup	Removable magnetic disks, optical discs, and tapes	10^{0}–10^{2}	10^{8}–10^{12}	10^{-5}–10^{-7}

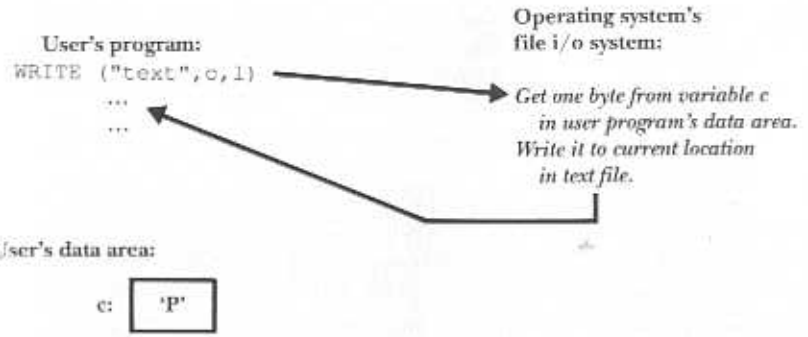


FIGURE 3.13 The WRITE( ) statement tells the operating system to send one character to disk and gives the operating system the location of the character. The operating system takes over the job of doing the actual writing and then returns control to the calling program.

3.5 A Journey of a Byte

What happens when a program writes a byte to a file on a disk? We know what the program does (it says WRITE(. . .)), and we now know something about how the byte is stored on a disk, but we haven't looked at what happens *between* the program and the disk. The whole story of what happens to data between program and disk is not one we can tell here, but we can give you an idea of the many different pieces of hardware and software involved and the many jobs that have to be done by looking at one example of a journey of one byte.

Suppose we want to append a byte representing the character '*P*' stored in a character variable *c* to a file named in the variable TEXT stored somewhere on a disk. From the program's point of view, the entire journey that the byte will take might be represented by the statement

```
WRITE(TEXT, c, 1)
```

but the journey is much longer than this simple statement suggests.

The *WRITE( )* statement results in a call to the computer's operating system, which has the task of seeing that the rest of the journey is completed successfully (Fig. 3.13). Often our program can provide the operating

system with information that helps it carry out this task more effectively, but once the operating system has taken over, the job of overseeing the rest of the journey is largely beyond our program's control.

3.5.1 The File Manager

An operating system is not a single program, but a collection of programs, each one designed to manage a different part of the computer's resources. Among these programs are ones that deal with file-related matters and I/O devices. We call this subset of programs the operating system's *file manager*. The file manager may be thought of as several layers of procedures (Fig. 3.14), with the upper layers dealing mostly with symbolic, or *logical*, aspects of file management, and the lower layers dealing more with the *physical* aspects. Each layer calls the one below it, until, at the lowest level, the byte is actually written to the disk.

The file manager begins by finding out whether the logical characteristics of the file are consistent with what we are asking it to do with the file. It may look up the requested file in a table, where it finds out such things as whether the file has been opened, what type of file the byte is being sent to (a binary file, a text file, some other organization), who the file's owner is, and whether WRITE( ) access is allowed for this particular user of the file.

The file manager must also determine where in the file TEXT the '*P*' is to be deposited. Since the '*P*' is to be appended to the file, the file manager needs to know where the end of the file is—the physical location of the last sector in the file. This information is obtained from the file allocation table (FAT) described earlier. From the FAT, the file manager locates the drive, cylinder, track, and sector where the byte is to be stored.

3.5.2 The I/O Buffer

Next, the file manager determines whether the sector that is to contain the '*P*' is already in RAM or needs to be loaded into RAM. If the sector needs to be loaded, the file manager must find an available *system I/O buffer* space for it, then read it from the disk. Once it has the sector in a buffer in RAM, the file manager can deposit the '*P*' into its proper position in the buffer (Fig. 3.15). The system I/O buffer allows the file manager to read and write data in sector-sized or block-sized units. In other words, it enables the file manager to ensure that the organization of data in RAM conforms to the organization it will have on the disk.

Instead of sending the sector immediately to the disk, the file manager usually waits to see if it can accumulate more bytes going to the same sector

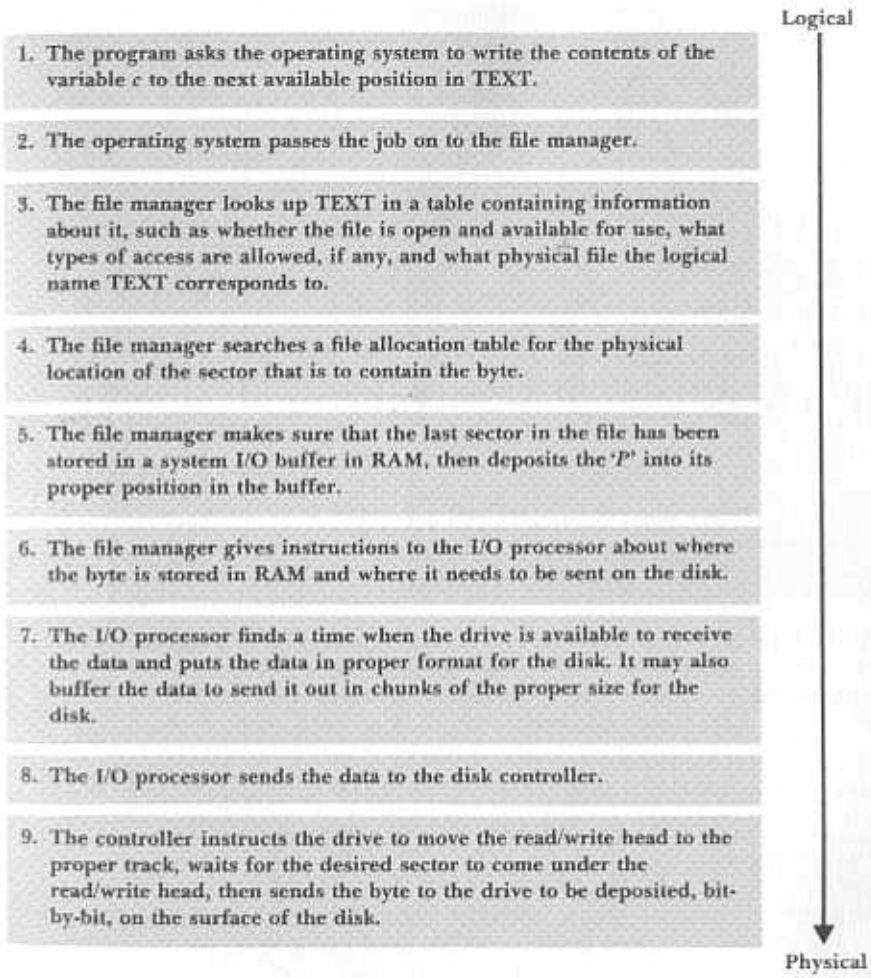


FIGURE 3.14 Layers of procedures involved in transmitting a byte from a program's data area to a file called TEXT on disk.

before actually transmitting anything. Even though the statement WRITE-(TEXT,c,1) seems to imply that our character is being sent immediately to the disk, it may in fact be kept in RAM for some time before it is sent. (There are many situations in which the file manager cannot wait until a buffer is filled before transmitting it. For instance, if TEXT were closed, it would have to *flush* all output buffers holding data waiting to be written to TEXT so the data would not be lost.)

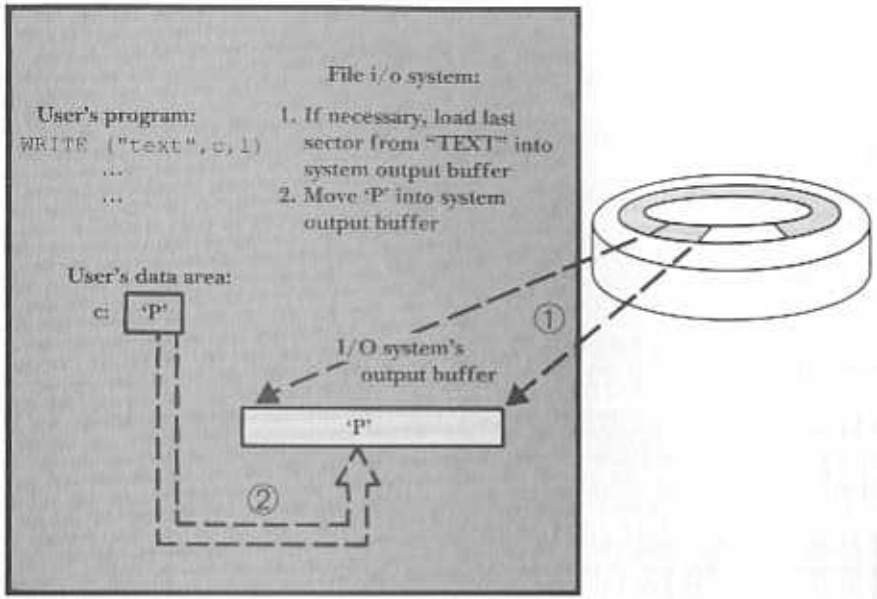


FIGURE 3.15 The file manager moves *P* from the program's data area to a system output buffer, where it may join other bytes headed for the same place on the disk. If necessary, the file manager may have to load the corresponding sector from the disk into the system output buffer.

3.5.3 The Byte Leaves RAM: The I/O Processor and Disk Controller

So far, all of our byte's activities have occurred within the computer's primary memory and have probably been carried out by the computer's central processing unit (CPU). The byte has travelled along data paths that are designed to be very fast and that are relatively expensive. Now it is time for the byte to travel along a data path that is likely to be slower and narrower than the one in primary memory. (A typical computer might have an internal data-path width of four bytes, whereas the width of the path leading to the disk might be only two bytes.)

Because of bottlenecks created by these differences in speed and data-path widths, our byte and its companions might have to wait for an external data path to become available. This also means that the CPU has extra time on its hands as it deals out information in small enough chunks and at slow enough speeds that the world outside can handle them. In fact, the differences between the internal and external speeds for transmitting

data are often so great that the CPU can transmit to several external devices simultaneously.

The processes of disassembling and assembling groups of bytes for transmission to and from external devices are so specialized that it is unreasonable to ask an expensive, general-purpose CPU to spend its valuable time doing I/O when a simpler device could do the job as well, freeing the CPU to do the work that it is most suited for. Such a special-purpose device is called an *I/O processor*.

An I/O processor may be anything from a simple chip capable of taking a byte and, on cue, just passing it on; to a powerful, small computer capable of executing very sophisticated programs and communicating with many devices simultaneously. The I/O processor takes its instructions from the operating system, but once it begins processing I/O, it runs independently, relieving the operating system (and the CPU) of the task of communicating with secondary storage devices. This allows I/O processes and internal computing to overlap.[†]

In a typical computer, the file manager might now tell the I/O processor that there is data in the buffer that is to be transmitted to the disk, how much data there is, and where it is to go on the disk. This information might come in the form of a little program that the operating system constructs and the I/O processor executes (Fig. 3.16).

The job of actually controlling the operation of the disk is done by a device called a *disk controller*. The I/O processor asks the disk controller if the disk drive is available for writing. If there is much I/O processing, there is a good chance that the drive will not be available and that our byte will have to wait in its buffer until the drive becomes available.

What happens next often makes the time spent so far seem insignificant in comparison: The disk drive is instructed to move its read/write head to the track and sector on the drive where our byte and its companions are to be stored. For the first time, a device is being asked to do something mechanical! The read/write head must seek to the proper track (unless it is already there), and then wait until the disk has spun around so the desired sector is under the head. Once the track and sector are located, the I/O processor (or perhaps the controller) can send out bytes, one at a time, to the drive. Our byte waits until its turn comes, then travels, alone, to the drive, where it probably is stored in a little one-byte buffer while it waits to be deposited on the disk.

[†]On many systems the I/O processor can take data directly from RAM, without further involvement from the CPU. This process is called *direct memory access* (DMA). On other systems, the CPU must place the data in special I/O registers before the I/O processor can have access to it.

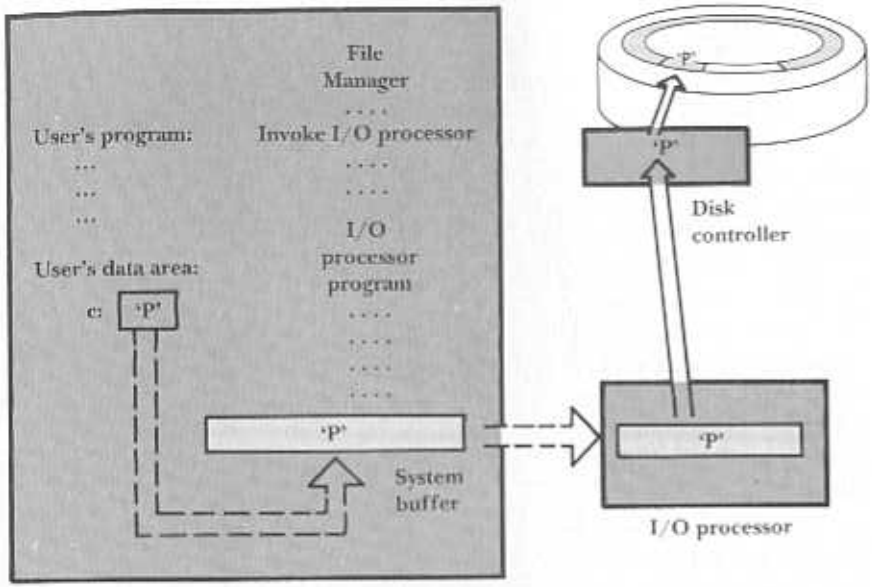


FIGURE 3.16 The file manager sends the I/O processor instructions in the form of an I/O processor program. The I/O processor gets the data from the system buffer, prepares it for storing on the disk, and then sends it to the disk controller, which deposits it on the surface of the disk.

Finally, as the disk spins under the read/write head, the eight bits of our byte are deposited, one at a time, on the surface of the disk (Fig. 3.16). There the '*P*' remains, at the end of its journey, spinning about at a leisurely 50 to 100 miles per hour.

3.6 Buffer Management

Any user of files can benefit from some knowledge of what happens to data travelling between a program's data area and secondary storage. One aspect of this process that is particularly important is the use of buffers. Buffering involves working with large chunks of data in RAM so the number of accesses to secondary storage can be reduced. We concentrate on the operation of *system* I/O buffers, but be aware that the use of buffers within programs can also substantially affect performance.

3.6.1 Buffer Bottlenecks

We know that a file manager allocates I/O buffers that are big enough to hold incoming data, but we have said nothing so far about *how many* buffers are used. In fact, it is common for file managers to allocate several buffers for performing I/O.

To understand the need for several system buffers, consider what happens if a program is performing both input and output on one character at a time, and only one I/O buffer is available. When the program asks for its first character, the I/O buffer is loaded with the sector containing the character, and the character is transmitted to the program. If the program then decides to output a character, the I/O buffer is filled with the sector into which the output character needs to go, destroying its original contents. Then when the next input character is needed, the buffer contents have to be written to disk to make room for the (original) sector containing the second input character, and so on.

Fortunately, there is a simple and generally effective solution to this ridiculous state of affairs, and that is to use more than one system buffer. For this reason, I/O systems almost always use at least two buffers—one for input and one for output.

Even if a program transmits data in only one direction, the use of a single system I/O buffer can slow it down considerably. We know, for instance, that the operation of reading a sector from a disk is extremely slow compared to the amount of time it takes to move data in RAM, so we can guess that a program that reads many sectors from a file might have to spend much of its time waiting for the I/O system to fill its buffer every time a read operation is performed before it can begin processing. When this happens, the program that is running is said to be *I/O bound*—the CPU spends much of its time just waiting for I/O to be performed. The solution to this problem is to use more than one buffer and to have the I/O system filling the next sector or block of data while the CPU is processing the current one.

3.6.2 Buffering Strategies

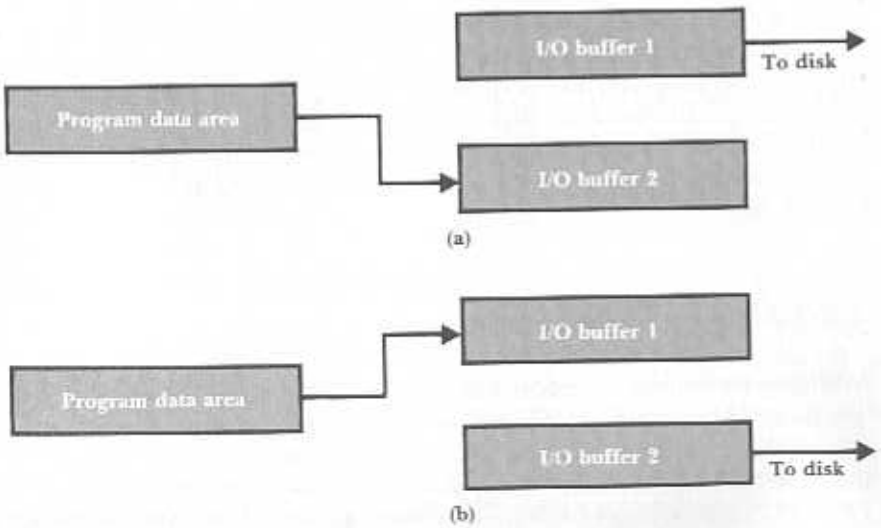
Multiple Buffering Suppose that a program is only writing to a disk and that it is I/O bound. The CPU wants to be filling a buffer at the same time that I/O is being performed. If *two* buffers are used and I/O-CPU overlapping is permitted, the CPU can be filling one buffer while the contents of the other are being transmitted to disk. When both tasks are

finished, the roles of the buffers can be exchanged. This technique of swapping the roles of two buffers after each output (or input) operation is called *double buffering*. Double buffering allows the operating system to be operating on one buffer while the other buffer is being loaded or emptied (Fig. 3.17).

The idea of swapping system buffers to allow processing and I/O to overlap need not be restricted to two buffers. In theory, any number of buffers can be used, and they can be organized in a variety of ways. The actual management of system buffers is usually done by the operating system and can rarely be controlled by programmers who do not work at the systems level. It is common, however, for users to be able to control the *number* of system buffers assigned to jobs.

Some file systems use a buffering scheme called *buffer pooling*: When a system buffer is needed, it is taken from a pool of available buffers and used. When the system receives a request to read a certain sector or block, it looks to see if one of its buffers already contains that sector or block. If no buffer contains it, then the system finds from its pool of buffers one that is not currently in use and loads the sector or block into it.

FIGURE 3.17 Double buffering: (a) The contents of system I/O buffer 1 are sent to disk while I/O buffer 2 is being filled; and (b) the contents of buffer 2 are sent to disk while I/O buffer 1 is being filled.



Several different schemes are used to decide which buffer to take from a buffer pool. One generally effective strategy is to take from the pool that buffer that is *least recently used*. When a buffer is accessed, it is put on a least-recently-used queue, so it is allowed to retain its data until all other less-recently-used buffers have been accessed. The least-recently-used (LRU) strategy for replacing old data with new data has many applications in computing. It is based on the assumption that a block of data that has been used recently is more likely to be needed in the near future than one that has been used less recently. (We encounter LRU again in later chapters.)

It is difficult to predict the point at which the addition of extra buffers ceases to contribute to improved performance. As the cost of RAM continues to decrease, so does the cost of using more and bigger buffers. On the other hand, the more buffers there are, the more time it takes for the file system to manage them. When in doubt, consider experimenting with different numbers of buffers.

Move Mode and Locate Mode Sometimes it is not necessary to distinguish between a program's data area and system buffers. When data must always be copied from a system buffer to a program buffer (or vice versa), the amount of time taken to perform the move can be substantial. This way of handling buffered data is called *move mode*, since it involves moving chunks of data from one place in RAM to another before they can be accessed.

There are two ways that move mode can be avoided. If the file manager can perform I/O directly between secondary storage and the program's data area, no extra move is necessary. Alternatively, the file manager could use system buffers to handle all I/O, but provide the program with the *locations*, through the use of pointer variables, of the system buffers. Both techniques are examples of a general approach to buffering called *locate mode*. When locate mode is used, a program is able to operate directly on data in the I/O buffer, eliminating the need to transfer data between an I/O buffer and a program buffer.

Scatter/Gather I/O Suppose you are reading in a file with many blocks, where each block consists of a header followed by data. You would like to put the headers in one buffer and the data in a different buffer so the data can be processed as a single entity. The obvious way to do this is to read the whole block into a single big buffer, and then move the different parts to their own buffers. Sometimes we can avoid this two-step process using a technique called *scatter input*. With scatter input, a single READ call

identifies not one, but a collection of buffers into which data from a single block is to be scattered.

The converse of scatter input is *gather output*. With gather output, several buffers can be gathered and written with a single WRITE call, avoiding the need to copy them to a single output buffer. When the cost of copying several buffers into a single output buffer is high, scatter/gather can have a significant effect on the running time of a program.

It is not always obvious when features like scatter/gather, locate mode, and buffer pooling are available in an operating system. You often have to go looking for them. Sometimes you can invoke them by communicating with your operating system, and sometimes you can cause them to be invoked by organizing your program in ways that are compatible with the way the operating system does I/O. Throughout this text we return many times to the issue of how to enhance performance by thinking about how buffers work and adapting programs and file structures accordingly.

3.7 I/O in UNIX

We see in the journey of a byte that we can view I/O as proceeding through several layers. UNIX provides a good example of how these layers occur in a real operating system, so we conclude this chapter with a look at UNIX. It is of course beyond the scope of this text to describe the UNIX I/O layers in detail. Rather, our objective here is just to pick a few features of UNIX that illustrate points made in the text. A secondary objective is to familiarize you with some of the important terminology used in describing UNIX systems. For a comprehensive, detailed look at how UNIX works, plus a thorough discussion of the design decisions involved in creating and improving UNIX, see Leffler et al. (1989).

3.7.1 The Kernel

In Fig. 3.14 we see how the process of transmitting data from a program to an external device can be described as proceeding through a series of layers. The topmost layer deals with data in *logical*, structural terms. We store in a file a name, a body of text, an image, an array of numbers, or some other logical entity. This reflects the view that an application has of what goes into a file. The layers that follow collectively carry out the task of turning the logical object into a collection of bits on a *physical* device.

Likewise, the topmost I/O layer in UNIX deals with data primarily in logical terms. This layer in UNIX consists of *processes* that impose certain logical views on files. Processes are associated with solving some problem,

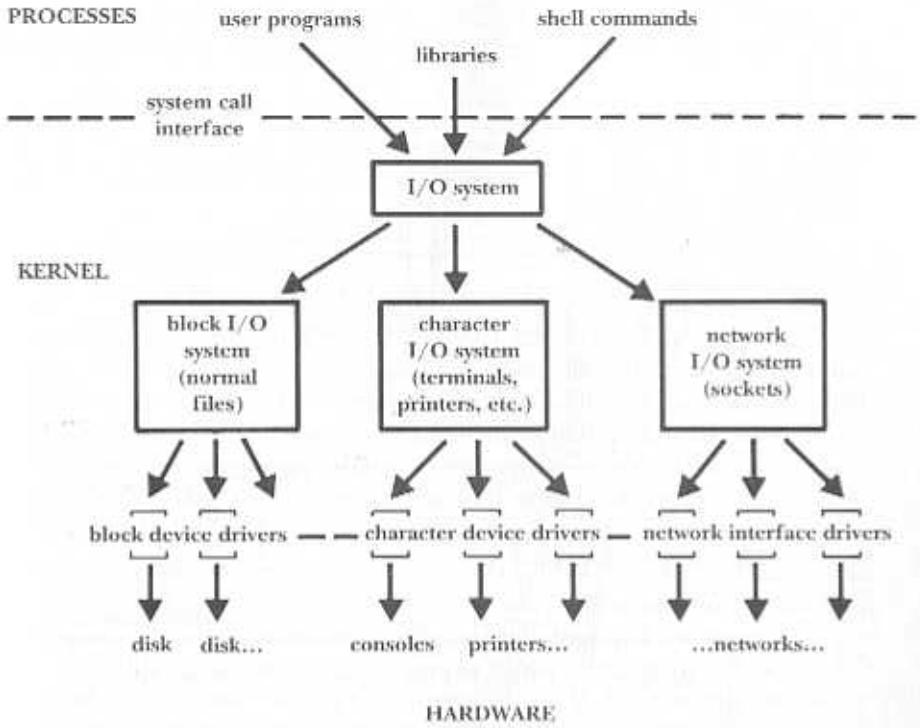


FIGURE 3.18 Kernel I/O structure.

such as counting the words in the file or searching for somebody's address. Processes include *shell routines* like cat and tail, *user programs* that operate on files, and *library routines* like scanf( ) and fread( ) that are called from programs to read strings, numbers, etc.

Below this layer is the UNIX *kernel*, which incorporates all the rest of the layers.† The components of the kernel that do I/O are illustrated in Fig. 3.18. The kernel views all I/O as operating on a sequence of bytes, so once we pass control to the kernel all assumptions about the logical view of a file are gone. The decision to design UNIX in this way—to make all operations below the top layer independent of an application's logical view of a file—is unusual. It is also one of the main attractions in choosing UNIX as a focus for this text, for UNIX lets us make all of the decisions

†It is beyond the scope of this text to describe the UNIX kernel in detail. For a full description of the UNIX kernel, including the I/O system, see Leffler et al. (1989).

about the logical structure of a file, imposing no restrictions on how we think about the file beyond the fact that it must be built from a sequence of bytes.

Let's illustrate the journey of a byte through the kernel, as we did earlier in this chapter by tracing the results of an I/O statement. We assume in this example that we are writing a character to disk. This corresponds to the left branch of the I/O system in Fig. 3.18.

When your program executes a system call such as

```
write (fd, &c, 1);
```

the kernel is invoked immediately.† The routines that let processes communicate directly with the kernel make up the *system call interface*. In this case, the system call instructs the kernel to write a character to a file.

The kernel I/O system begins by connecting the file descriptor *(fd)* in your program to some file or device in the filesystem. It does this by proceeding through a series of four tables that enable the kernel to find its way from a process to the places on the disk that will hold the file that they refer to. The four tables are

- a file descriptor table;
- an open file table, with information about open files;
- a file allocation table, which is part of a structure called an index node; and
- a table of index nodes, with one entry for each file in use.

Although these tables are managed by the kernel's I/O system, they are, in a sense, "owned" by different parts of the system:

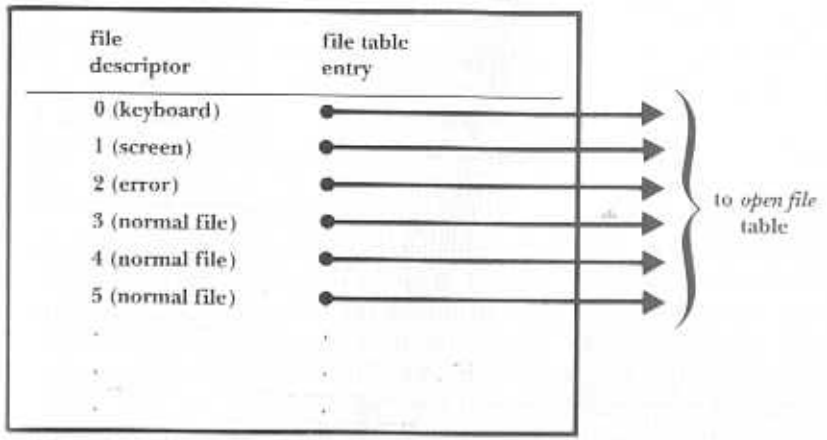
- The file descriptor table is owned by the process (your program).
- The open file table and index node tables are owned by the kernel.
- The index node itself is part of the filesystem.

The four tables are invoked in turn by the kernel to get the information it needs to write to your file on disk. Let's see how this works by looking at the functions of the tables.

The *file descriptor table* (Fig. 3.19a) is a simple table that associates each of the file descriptors used by a process with an entry in another table, the open file table. Every process has its own descriptor table, which includes entries for all files it has opened, including the "files" *STDIN*, *STDOUT*, and *STDERR*.

†This should not be confused with a *library* call, such as *fprintf( )*, which invokes the standard library to perform some additional operations on the data, such as converting it to an ASCII format, and *then* makes a corresponding system call.

(a) *descriptor* table



(b) *open file* table

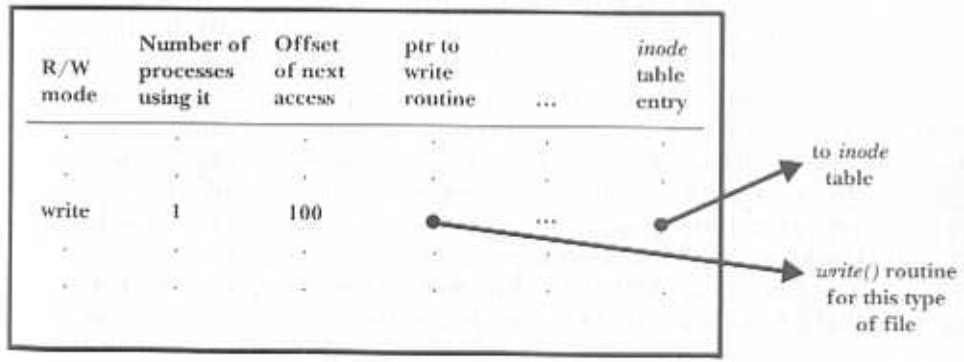


FIGURE 3.19 Descriptor table and open file table.

The *open file* table (Fig. 3.19b) contains entries for every open file. Every time a file is opened or created, a new entry is added to the open file table. These entries are called *file structures*, and they contain important information about how the corresponding file is to be used, such as the read/write mode used when it was opened, the number of processes currently using it, and the offset within the file to be used for the next read or write. The open file table also contains an array of pointers to generic

functions that can be used to operate on the file. These functions will differ depending on the type of file.

It is possible for several different processes to refer to the same open file table entry, so one process could read part of a file, another process could read the next part, and so forth, with each process taking over where the previous one stopped. On the other hand, if the same file is opened by two separate *open ( )* statements, two separate entries are made in the table, and the two processes operate on the file quite independently.[†]

The information in the open file table is transitory. It tells the kernel what it can do with a file that has been opened in a certain way and provides information on how it can operate on the file. The kernel still needs more information about the file itself, such as where the file is stored on disk, how big the file is, and who owns it. This information is found in an *index node*, more commonly referred to as an *inode* (Fig. 3.20).

An inode is a more permanent structure than an open file table's file structure. A file structure exists only while a file is open for access, but an inode exists as long as its corresponding file exists. For this reason, a file's inode is kept on disk *with* the file (though not physically adjacent to the file). When a file is opened, a copy of its inode is usually loaded into RAM where it is added to the aforementioned *inode table* for rapid access.

For the purposes of our discussion, the most important component of the inode is a list (index) of the disk blocks that make up the file. This list is the UNIX counterpart to the *file allocation table* that we described earlier in this chapter.[‡] Once the kernel's I/O system has the inode information, it knows all that it needs to know about the file. It then invokes an I/O processor program that is appropriate for the type of data, the type of operation, and the type of device that is to be written. In UNIX, this program is called a *device driver*.

The device driver sees that your data is moved from its buffer to its proper place on disk. Before we look at the role of device drivers in UNIX, it is instructive to look at how the kernel distinguishes among the different kinds of file data that it must deal with.

3.7.2 Linking File Names to Files

It is instructive to look a little more closely at how a file name is actually linked to the corresponding file. All references to files begin with a

[†]Of course, there are risks in letting this happen. If you are writing to a file with one process at the same time that you are independently reading from the file with another, the meaning of these may be difficult to determine.

[‡]This might not be a simple linear array. To accommodate both large and small files, this table often has a dynamic, tree-like structure.

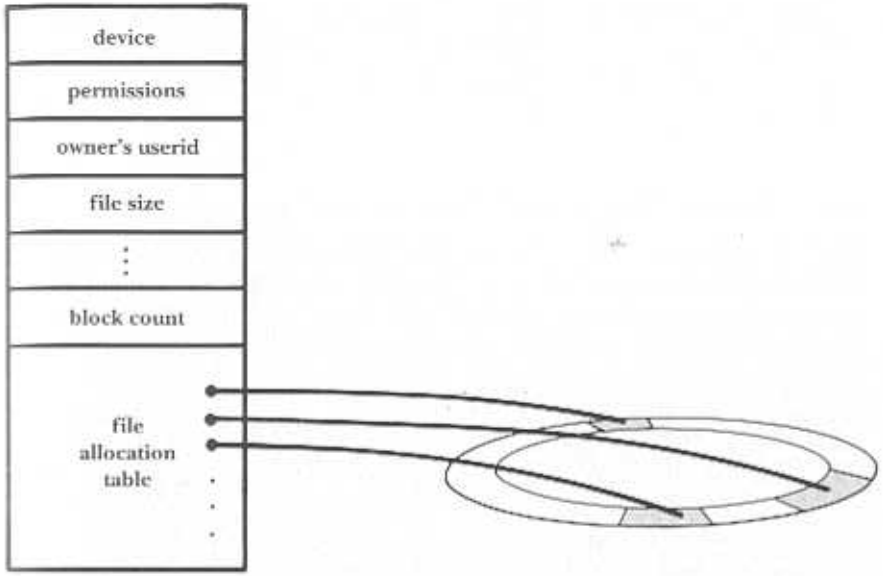


FIGURE 3.20 An inode. The inode is the data structure used by UNIX to describe the file. It includes the device containing the file, permissions, owner and group IDs, and file allocation table, among other things.

directory, for it is in directories that file names are kept. In fact, a directory is just a small file that contains, for each file, a file name together with a pointer to the file's inode on disk.[†] This pointer from a directory to the inode of a file is called a *hard link*. It provides a direct reference from the file name to all other information about the file. When a file is opened, this hard link is used to bring the inode into RAM and to set up the corresponding entry in the open file table.

It is possible for several file names to point to the same inode, so one file can have several different names. A field in the inode tells how many hard links there are to the inode. This means that if a file name is deleted and there are other file names for the same file, the file itself is not deleted; its inode's hard-link count is just decremented by one.

There is another kind of link, called a *soft link*, or *symbolic link*. A symbolic link links a file name to another file name, rather than to an actual file. Instead of being a pointer to an inode, a soft link is a pathname of some

[†]The actual structure of a directory is a little more complex than this, but these are the essential parts. See Leffler, et al. (1989) for details.

file. Since a symbolic link does not point to an actual file, it can refer to a directory or even to a file in a different file system. Symbolic links are not supported on all UNIX systems. UNIX System 4.3BSD supports symbolic links, but System V does not.

3.7.3 Normal Files, Special Files, and Sockets

The "everything is a file" concept in UNIX works only when we recognize that some files are quite a bit different from others. We see in Fig. 3.18 that the kernel distinguishes among three different types of files. *Normal files* are the files that this text is about. *Special files* almost always represent a stream of characters and control signals that drive some device, such as a line printer or a graphics device. The first three file descriptors in the descriptor table (Fig. 3.19a) are special files. *Sockets* are abstractions that serve as endpoints for interprocess communication.

At a certain conceptual level, these three different types of UNIX files are very similar, and many of the same routines can be used to access any of them. For instance, you can establish access to all three types by opening them, and you can write to them with the *write( )* system call.

3.7.4 Block I/O

In Fig. 3.18, we see that the three different types of files access their respective devices via three different I/O systems, the *block I/O system*, the *character I/O system*, and the *network I/O system*. Henceforth we ignore the second and third categories, since it is normal file I/O that we are most concerned with in this text.[†]

The block I/O system is the UNIX counterpart of the file manager in the journey of a byte. It concerns itself with how to transmit normal file data, viewed by the user as a sequence of bytes, onto a block-oriented device like a disk or tape. Given a byte to store on a disk, for example, it arranges to read in the sector containing the byte to be replaced, to replace the byte, and to write the sector back to the disk.

The UNIX view of a block device most closely resembles that of a disk. It is a randomly addressable array of fixed blocks. Originally all blocks were 512 bytes, which was the common sector size on most disks. No other organization (such as clusters) was imposed on the placement of files on

[†]This is not entirely true. Sockets, for example, can be used to move normal files from place to place. In fact, high-performance network systems bypass the normal file system in favor of sockets to squeeze every bit of performance out of the network.

disk. (In section 3.1.7 we saw how the design of later UNIX systems dealt with this convention.)

3.7.5 Device Drivers

For each peripheral device there is a separate set of routines, called a *device driver*, that performs the actual I/O between the I/O buffer and the device. A device driver is roughly equivalent to the I/O processor program described in the journey of a byte.

Since the block I/O system views a peripheral device as an array of physical blocks, addressed as block 0, block 1, etc., a block I/O device driver's job is to take a block from a buffer, destined for one of these physical blocks, and see that it gets deposited in the proper physical place on the device. This saves the block I/O part of the kernel from having to know anything about the specific device it is writing to, other than its identity and that it is a block device. A thorough discussion of device drivers for block, character, and network I/O can be found in Leffler et al. (1989).

3.7.6 The Kernel and Filesystems

In Chapter 2 we described the UNIX concept of a *filesystem*. A UNIX filesystem is a collection of files, together with secondary information about the actual files in the system. A filesystem includes the directory structure, the directories, ordinary files, and the inodes that describe the files.

In our discussions we talk about the filesystem as if it is part of the kernel's I/O system, which it is, but it is also in a sense separate from it. All parts of a filesystem reside on disk, rather than in RAM where the kernel does its work. These parts are brought into RAM by the kernel as needed. This separation of the filesystem from the kernel has many advantages. One important advantage is that we can tune a filesystem to a particular device or usage pattern independently of how the kernel views files. The discussions in section 3.1.7 of 4.3BSD block organization are file-system concerns, for example, and need not have any effect on how the kernel works.

Another advantage of keeping the filesystem and I/O system distinct is that we can have separate filesystems that are organized differently, perhaps on different devices, but are accessible by the same kernel. In Appendix A, for instance, we describe the design of a filesystem on CDROM that is organized quite differently from a typical disk-based file system yet looks just like any other filesystem to the user and to the I/O system.

3.7.7 Magnetic Tape and UNIX

Important as it is to computing, magnetic tape is somewhat of an orphan in the UNIX view of I/O. A magnetic tape unit has characteristics similar to both block I/O devices (being block oriented) and character devices (being primarily used for sequential access), but does not fit nicely into either category. Character devices read and write streams of data, not blocks, and block devices in general access blocks randomly, not sequentially.

Since block I/O is generally the least inappropriate of the two inappropriate paradigms for tape, a tape device is normally considered in UNIX to be a block I/O device and hence is accessed through the block I/O interface. But because the block I/O interface is most often used to write to random-access devices, disks, it does not require blocks to be written in sequence, as they must be written to a tape. This problem is solved by allowing only one write request at a time per tape drive. When high-performance I/O is required, the character device interface can be used in a raw mode to stream data to tapes, bypassing the stage that requires the data to be collected into relatively small blocks before or after transmission.

SUMMARY

In this chapter we look at the software environment in which file processing programs must operate and at some of the hardware devices on which files are commonly stored, hoping to understand how they influence the ways we design and process files. We begin by looking at the two most common storage media: magnetic disks and tapes.

A disk drive consists of a set of read/write heads that are interspersed among one or more platters. Each platter contributes one or two surfaces, each surface contains a set of concentric tracks, and each track is divided into sectors or blocks. The set of tracks that can be read without moving the read/write heads is called a cylinder.

There are two basic ways to address data on disks: by sector and by block. Used in this context, the term *block* refers to a group of records that are stored together on a disk and treated as a unit for I/O purposes. When blocks are used, the user is better able to make the physical organization of data correspond to its logical organization, and hence can sometimes improve performance. Block-organized drives also sometimes make it possible for the disk drive to search among blocks on a track for a record with a certain key without first having to transmit the unwanted blocks into RAM.

Three possible disadvantages of block-organized devices are the danger of internal track fragmentation, the burden of dealing with the extra

complexity that the user has to bear, and the loss of opportunities to do some of the kinds of synchronization (such as sector interleaving) that sector-addressing devices provide.

The cost of a disk access can be measured in terms of the time it takes for seeking, rotational delay, and transfer time. If sector interleaving is used, it is possible to access logically adjacent sectors by separating them physically by one or more sectors. Although it takes much less time to access a single record directly than sequentially, the extra seek time required for doing direct accesses makes it much slower than sequential access when a series of records is to be accessed.

Despite increasing disk performance, network speeds have improved to the point that disk access is often a significant bottleneck in an overall I/O system. A number of techniques are available for addressing this problem, including striping, the use of RAM disks, and disk caching.

Research done in connection with BSD UNIX shows that block size can have a major effect on performance. By increasing the default block size from 512 bytes to 4,096 bytes, throughput was improved enormously, especially for large files, because eight times as much data could be transferred in a single access. A negative consequence of this reorganization was that wasted storage increased from 6.9% for 512-byte blocks to 45.6% for 4,096-byte blocks. It turned out that this problem of wasted space could be dealt with by treating the 4,096-byte blocks as clusters of 512-byte blocks, which could be allocated to different files.

Though not as important as disks, magnetic tape has an important niche in file processing. Tapes are inexpensive, reasonably fast for sequential processing, compact, robust, and easy to store and transport. Data are usually organized on tapes in one-bit-wide parallel tracks, with a bit-wide cross-section of tracks interpreted as one or more bytes. When estimating processing speed and space utilization, it is important to recognize the role played by the interblock gap. Effective recording density and effective transmission rate are useful measurements of the performance one can expect to achieve for a given physical file organization.

In comparing disk and tape as secondary storage media, we see that disks are replacing tape in more and more cases. This is largely because RAM is becoming less expensive, relative to secondary storage, which means that one of the earlier advantages of tape over disk, the ability to do sequential access without seeking, has diminished significantly.

This chapter follows a journey of a byte as it is sent from RAM to disk. The journey involves the participation of many different programs and devices, including

- a user's program, which makes the initial call to the operating system;

- the operating system's file manager, which maintains tables of information that it uses to translate between the program's logical view of the file and the physical file where the byte is to be stored;
- an I/O processor and its software, which transmit the byte, synchronizing the transmission of the byte between an I/O buffer in RAM and the disk;
- the disk controller and its software, which instruct the drive about how to find the proper track and sector, then send the byte; and
- the disk drive, which accepts the byte and deposits it on the disk surface.

Next, we take a closer look at buffering, focusing mainly on techniques for managing buffers to improve performance. Some techniques include double buffering, buffer pooling, locate-mode buffering, and scatter/gather buffering.

We conclude with a second look at I/O layers, this time concentrating on UNIX. We see that every I/O system call begins with a call to the UNIX kernel, which knows nothing about the logical structure of a file, treating all data essentially the same—as a sequence of bytes to be transmitted to some external device. In doing its work the I/O system in the kernel invokes four tables: a file descriptor table, an open file table, an inode table, and a file access table in the file's inode. Once the kernel has determined which device to use and how to access it, it calls on a device driver to carry out the actual accessing.

Although it treats every file as a sequence of bytes, the kernel I/O system deals differently with three different types of I/O: block I/O, character I/O, and network I/O. In this text we concentrate on block I/O. We look briefly at the special role of the file system within the kernel, describing how it uses links to connect file names in directories to their corresponding inodes. Finally, we remark on the reasons that magnetic tape does not fit well into the UNIX paradigm for I/O.

KEY TERMS

bpi. Bits per inch per track. On a disk, data is recorded serially on tracks. On a tape, data are recorded in parallel on several tracks, so a 6,250-bpi nine-track tape contains 6,250 bytes per inch, when all nine tracks are taken into account (one track being used for parity).

Block. Unit of data organization corresponding to the amount of data transferred in a single access. *Block* often refers to a collection of records, but it may be a collection of sectors (see *cluster*) whose size has no correspondence to the organization of the data. A block is sometimes called a physical record; a sector is sometimes called a block.

Block device. In UNIX, a device such as a disk drive that is organized in blocks and accessed accordingly.

Block I/O. I/O between a computer and a block device.

Block organization. Disk drive organization that allows the user to define the size and organization of blocks, and then access a block by giving its block address or the key of one of its records. (See *sector organization.*)

Blocking factor. The number of records stored in one block.

Character device. In UNIX, a device such as a keyboard or printer (or tape drive when stream I/O is used) that sends or receives data in the form of a stream of characters.

Character I/O. I/O between a computer and a character device.

Cluster. Minimum unit of space allocation on a sectored disk, consisting of one or more contiguous sectors. The use of large clusters can improve sequential access times by guaranteeing the ability to read longer spans of data without seeking. Small clusters tend to decrease internal fragmentation.

Controller. Device that directly controls the operation of one or more secondary storage devices, such as disk drives and magnetic tape units.

Count subblock. On block-organized drives, a small block that precedes each data block and contains information about the data block, such as its byte count and its address.

Cylinder. The set of tracks on a disk that are directly above and below each other. All of the tracks in a given cylinder can be accessed without having to move the access arm; that is, they can be accessed without the expense of seek time.

Descriptor table. In UNIX, a table associated with a single process that links all of the file descriptors generated by that process to corresponding entries in an open file table.

Device driver. In UNIX, an I/O processor program invoked by the kernel that performs I/O for a particular device.

Direct access storage device (DASD). Disk or other secondary storage device that permits access to a specific sector or block of data without first requiring the reading of the blocks that precede it.

Direct memory access (DMA). Transfer of data directly between RAM and peripheral devices, without significant involvement by the CPU.

Disk cache. A segment of RAM configured to contain pages of data from a disk. Disk caches can lead to substantial improvements in access time when access requests exhibit a high degree of locality.

Disk pack. An assemblage of magnetic disks mounted on the same vertical shaft. A pack of disks is treated as a single unit consisting of a number of cylinders equivalent to the number of tracks per surface. If disk packs are removable, different packs can be mounted on the same drive at different times, providing a convenient form of offline storage for data that can be accessed directly.

Effective recording density. Recording density after taking into account the space used by interblock gaps, nondata subblocks, and other space-consuming items that accompany data.

Effective transmission rate. Transmission rate after taking into account the time used to locate and transmit the block of data in which a desired record occurs.

Extent. One or more adjacent clusters allocated as part (or all) of a file. The number of extents in a file reflects how dispersed the file is over the disk. The more dispersed a file, the more seeking must be done in moving from one part of the file to another.

File allocation table (FAT). A table that contains mappings to the physical locations of all the clusters in all files on disk storage.

File manager. The part of an operating system that is responsible for managing files, including a collection of programs whose responsibilities range from keeping track of files to invoking I/O processes that transmit information between primary and secondary storage.

File structure. In connection with the open file table in a UNIX kernel, the term *file structure* refers to a structure that holds information the kernel needs about an open file. File structure information includes such things as the file's read/write mode, number of processes currently using it, and the offset within the file to be used for the next read or write.

Filesystem. In UNIX, a hierarchical collection of files, usually kept on a single secondary device, such as a hard disk or CD-ROM.

Fixed disk. A disk drive with platters that may not be removed.

Formatting. The process of preparing a disk for data storage, involving such things as laying out sectors, setting up the disk's file allocation table, and checking for damage to the recording medium.

Fragmentation. Space that goes unused within a cluster, block, track, or other unit of physical storage. For instance, track fragmentation occurs when space on a track goes unused because there is not enough space left to accommodate a complete block.

Frame. A one-bit-wide slice of tape, usually representing a single byte.

Hard link. In UNIX, an entry in a directory that connects a file name to the inode of the corresponding file. There can be several hard links to a single file; hence a file can have several names. A file is not deleted until all hard links to the file are deleted.

Index node. In UNIX, a data structure associated with a file that describes the file. An index node includes such information as a file's type, its owner and group IDs, and a list of the disk blocks that comprise the file. A more common name for index node is *inode*.

Inode. See *index node*.

Interblock gap. An interval of blank space that separates sectors, blocks, or subblocks on tape or disk. In the case of tape, the gap provides sufficient space for the tape to accelerate or decelerate when starting or stopping. On both tapes and disks the gaps enable the read/write heads to tell accurately when one sector (or block or subblock) ends and another begins.

Interleaving factor. Since it is often not possible to read physically adjacent sectors of a disk, logically adjacent sectors are sometimes arranged so they are not physically adjacent. This is called interleaving. The interleaving factor refers to the number of physical sectors the next logically adjacent sector is located from the current sector being read or written.

I/O processor. A device that carries out I/O tasks, allowing the CPU to work on non-I/O tasks.

Kernel. The central part of the UNIX operating system.

Key subblock. On block-addressable drives, a block that contains the key of the last record in the data block that follows it, allowing the drive to search among the blocks on a track for a block containing a certain key, without having to load the blocks into primary memory.

Mass storage system. General term applied to storage units with large capacity. Also applied to very high-capacity secondary storage systems that are capable of transmitting data between a disk and any of several thousand tape cartridges within a few seconds.

Nominal recording density. Recording density on a disk track or magnetic tape without taking into account the effects of gaps or nondata subblocks.

Nominal transmission rate. Transmission rate of a disk or tape unit without taking into account the effects of such extra operations as seek time for disks and interblock gap traversal time for tapes.

Open file table. In UNIX, a table owned by the kernel with an entry, called a file structure, for each open file. See *file structure*.

Parity. An error-checking technique in which an extra parity bit accompanies each byte and is set in such a way that the total number of 1 bits is even (even parity) or odd (odd parity).

Platter. One disk in the stack of disks on a disk drive.

Process. An executing program. In UNIX, several instances of the same program can be executing at the same time, as separate processes. The kernel keeps a separate file descriptor table for each process.

RAM disk. Block of RAM configured to simulate a disk.

Rotational delay. The time it takes for the disk to rotate so the desired sector is under the read/write head.

Scatter/gather I/O. Buffering techniques that involve, on input, scattering incoming data into more than one buffer, and, on output, gathering data from several buffers to be output as a single chunk of data.

Sector. The fixed-sized data blocks that together make up the tracks on certain disk drives. Sectors are the smallest addressable unit on a disk whose tracks are made up of sectors.

Sector organization. Disk drive organization that uses sectors.

Seek time. The time required to move the access arm to the correct cylinder on a disk drive.

Sequential access device. A device, such as a magnetic tape unit or card reader, in which the medium (e.g., tape) must be accessed from the beginning. Sometimes called a serial device.

Socket. In UNIX, a socket is an abstraction that serves as an endpoint of communication within some domain. For example, a socket can be used to provide direct communication between two computers. Although in some ways the kernel treats sockets like files, we do not deal with sockets in this text.

Soft link. See *symbolic link*.

Special file. In UNIX, the term *special file* refers to a stream of characters and control signals that drive some device, such as a line printer or a graphics device.

Streaming tape drive. A tape drive whose primary purpose is dumping large amounts of data from disk to tape or from tape to disk.

Subblock. When blocking is used, there are often separate groupings of information concerned with each individual block. For example, a count subblock, a key subblock, and a data subblock might all be present.

Symbolic link. In UNIX, an entry in a directory that gives the pathname of a file. Since a symbolic link is an indirect pointer to a file, it is not as closely associated with the file as a hard link. Symbolic links can point to directories, or even to files in other filesystems.

Track. The set of bytes on a single surface of a disk that can be accessed without seeking (without moving the access arm). The surface of a disk can be thought of as a series of concentric circles, with each circle corresponding to a particular position of the access arm and read/write heads. Each of these circles is a track.

Transfer time. Once the data we want is under the read/write head, we have to wait for it to pass under the head as we read it. The amount of time required for this motion and reading is the transfer time.

EXERCISES

1. Determine as well as you can what the journey of a byte would be like on your system. You may have to consult technical reference manuals that describe your computer's file management system, operating system, and peripheral devices. You may also want to talk to local gurus who have experience using your system.

2. Suppose you are writing a list of names to a text file, one name per write statement. Why is it not a good idea to close the file after every write, and then reopen it before the next write?

3. Find out what utility routines are available on your computer system for monitoring I/O performance and disk utilization. If you have a large computing system, there are different routines available for different kinds of users, depending on what privileges and responsibilities they have.

4. When you create or open a file in C or Pascal, you must provide certain information to your computer's file manager so it can handle your file properly. Compared to certain languages, such as PL/I or COBOL, the amount of information you must provide in C or Pascal is very small. Find a text or manual on PL/I or COBOL and look up the ENVIRONMENT file description attribute, which can be used to tell the file manager a great deal about how you expect a file to be organized and used. Compare PL/I or COBOL with C or Pascal in terms of the types of file specifications available to the programmer.

5. Much is said in section 3.1 about how disk space is organized physically to store files. Assume that no such complex organization is used and that every file must occupy a single contiguous piece of a disk, somewhat the way a file is stored on tape. How does this simplify disk storage? What problems does it create?

6. A disk drive uses 512-byte sectors. If a program requests that a 128-byte record be written to disk, the file manager may have to read a sector from the disk before it can write the record. Why? What could you do to decrease the number of times such an extra read is likely to occur?

7. We have seen that some disk operating systems allocate storage space on disks in clusters and/or extents, rather than sectors, so the size of any file must be a multiple of a cluster or extent.
   a. What are some advantages and potential disadvantages of this method of allocating disk space?
   b. How appropriate would the use of large extents be for an application that mostly involves sequential access of very large files?
   c. How appropriate would large extents be for a computing system that serves a large number of C programmers? (C programs tend to be small, so there are likely to be many small files that contain C programs.)
   d. The VAX record management system uses a default cluster size of three 512-byte sectors but lets a user reformat a drive with any cluster size from 1 to 65,535 sectors. When might a cluster size larger than three sectors be desirable? When might a smaller cluster size be desirable?

8. In early UNIX systems, inodes were kept together on one part of a disk, while the corresponding data was scattered elsewhere on the disk. Later editions divided disk drives into groups of adjacent cylinders called cylinder groups, in which each cylinder group contains inodes and their corresponding data. How does this new organization improve performance?

9. In early UNIX systems, the minimum block size was 512 bytes, with a cluster size of one. The block size was increased to 1,024 bytes in 4.0BSD, more than doubling its throughput. Explain how this could occur.

10. Draw pictures that illustrate the role of fragmentation in determining the numbers in Table 3.2, section 3.1.7.

11. The IBM 3350 disk drive uses block addressing. The two subblock organizations described in the text are available:
- Count-data, where the extra space used by count subblock and interblock gaps is equivalent to 185 bytes; and
- Count-key-data, where the extra space used by the count and key subblocks and accompanying gaps is equivalent to 267 bytes, plus the key size.

An IBM 3350 has 19,069 usable bytes available per track, 30 tracks per cylinder, and 555 cylinders per drive. Suppose you have a file with 350,000 80-byte records that you want to store on a 3350 drive. Answer the following questions. Unless otherwise directed, assume that the blocking factor is 10 and that the count-data subblock organization is used.
   a. How many blocks can be stored on one track? How many records?
   b. How many blocks can be stored on one track if the count-key-data subblock organization is used and key size is 13 bytes?
   c. Make a graph that shows the effect of block size on storage utilization, assuming count-data subblocks. Use the graph to help predict the best and worst possible blocking factor in terms of storage utilization.
   d. Assuming that access to the file is always sequential, use the graph from the preceding question to predict the best and worst blocking factor. Justify your answer in terms of efficiency of storage utilization and processing time.
   e. How many cylinders are required to hold the file (blocking factor 10 and count-data format)? How much space will go unused due to internal track fragmentation?
   f. If the file were stored on contiguous cylinders and if there were no interference from other processes using the disk drive, the average seek time for a random access of the file would be about 12 msec. Use this rate to compute the average time needed to access one record randomly.
   g. Explain how retrieval time for random accesses of records is affected by increasing block size. Discuss trade-offs between storage efficiency and retrieval when different block sizes are used. Make a table with different block sizes to illustrate your explanations.
   h. Suppose the file is to be sorted and a shell sort is to be used to sort the file. Since the file is too large to read into memory, it will be sorted in place, on the disk. It is estimated (Knuth, 1973b, p. 380) that this requires about $15N1.25$ moves of records, where N represents the total number of records in the file. Each move requires a random access. If all of the preceding is true, how long does it take to sort the file? (As you will see, this is not a very good solution. We provide much better ones in Chapter 7, which deals with cosequential processing.)

12. A sectored disk drive differs from one with a block organization in that there is less of a correspondence between the logical and physical organization of data records or blocks.

For example, consider the Digital RM05 disk drive, which uses sector addressing. It has 32 512-byte sectors per track, 19 tracks per cylinder, and 823 cylinders per drive. From the drive's (and drive controller's) point of

view, a file is just a vector of bytes divided into 512-byte sectors. Since the drive knows nothing about where one record ends and another begins, a record can span two or more sectors, tracks, or cylinders.

One common way that records are formatted on the RM05 is to place a two-byte field at the beginning of each block, giving the number of bytes of data, followed by the data itself. There is no extra gap and no other overhead. Assuming that this organization is used, and that you want to store a file with 350,000 80-byte records, answer the following questions:

a. How many records can be stored on one track if one record is stored per block?
b. How many cylinders are required to hold the file?
c. How might you block records so each physical record access results in 10 actual records being accessed? What are the benefits of doing this?

13. Suppose you have a collection of 500 large images stored in files, one image per file, and you wish to "animate" these images by displaying them in sequence on a workstation at a rate of at least 15 images per second over a high-speed network. Your secondary storage consists of a disk farm with 30 disk drives, and your disk manager permits striping over as many as 30 drives, if you request it. Your drives are guaranteed to perform I/O at a steady rate of 2 megabytes per second. Each image is 3 megabytes in size. Network transmission speeds are not a problem.

a. Describe in broad terms the steps involved in doing such an animation in real time from disk.
b. Describe the performance issues that you have to consider in implementing the animation. Use numbers.
c. How might you configure your I/O system to achieve the desired performance?

14. Consider the 1,000,000-record mailing list file discussed in the text. The file is to be backed up on 2,400-foot reels of 6,250-bpi tape with 0.3-inch interblock gaps. Tape speed is 200 inches per second.

a. Show that only one tape would be required to back up the file if a blocking factor of 50 is used.
b. If a blocking factor of 50 is used, how many extra records could be accommodated on a 2,400-foot tape?
c. What is the effective recording density when a blocking factor of 50 is used?
d. How large does the blocking factor have to be to achieve the maximum effective recording density? What negative results can result from increasing the blocking factor? (*Note:* An I/O buffer large enough to hold a block must be allocated.)
e. What would be the minimum blocking factor required to fit the file onto the tape?
f. If a blocking factor of 50 is used, how long would it take to read one block, including the gap? What would the effective transmission rate be? How long would it take to read the entire file?
g. How long would it take to perform a binary search for one record in the file, assuming that it is not possible to read backwards on the tape? (Assume that it takes 60 seconds to rewind the tape.) Compare this with the expected average time it would take for a sequential search for one record.
h. We implicitly assume in our discussions of tape performance that the tape drive is always reading or writing at full speed, so no time is lost by starting and stopping. This is not necessarily the case. For example, some drives automatically stop after writing each block.

Suppose that the extra time it takes to start before reading a block and to stop after reading the block totals 1 msec, and that the drive must start before and stop after reading each block. How much will the effective transmission rate be decreased due to starting and stopping if the blocking factor is 1? What if it is 50?

15. Why are there interblock gaps on tapes? In other words, why do we not just jam all records into one block?

16. The use of large blocks can lead to severe internal fragmentation of tracks on disks. Does this occur when tapes are used? Explain.

FURTHER READINGS

Many textbooks contain more detailed information on the material covered in this chapter. In the area of operating systems and file management systems, we have found the operating system texts by Deitel (1984), Peterson and Silberschatz (1985), and Madnick and Donovan (1974) useful. Hanson (1982) has a great deal of material on blocking and buffering, secondary storage devices, and performance. Flores's book (1973) on peripheral devices may be a bit dated, but it contains a comprehensive treatment of the subject.

Bohl (1981) provides a thorough treatment of mainframe-oriented IBM DASDs. Chaney and Johnson (1984) wrote a good article on maximizing hard disk performance on small computers. Ritchie and Thompson (1974), Kernighan and Ritchie (1978), Deitel (1984), and McKusick et al. (1984) provide information on how file I/O is handled in the UNIX operating system. The latter provides a good case study of ways in which a filesystem can be altered to provide substantially faster throughput for certain applications. A comprehensive coverage of UNIX I/O from the design perspective can be found in Leffler et al. (1989).

Information on specific systems and devices can often be found in manuals and documentation published by manufacturers. (Unfortunately, information about how software actually works is often proprietary and therefore not available.) If you use a VAX, we recommend the manuals *Introduction to the VAX Record Management Services* (Digital, 1978), *VAX Software Handbook* (Digital, 1982), and *Peripherals Handbook* (Digital, 1981). UNIX users will find it useful to look at the Bell Laboratories' monograph *The UNIX I/O System* by Dennis Ritchie (1979). Users of IBM PCs will find the *Disk Operating System* (Microsoft, 1983 or later) manual useful.

4

Fundamental File Structure Concepts

CHAPTER OBJECTIVES

- Introduce file structure concepts dealing with
  - Stream files;
  - Field and record boundaries;
  - Fixed-length and variable-length fields and records;
  - Search keys and canonical forms;
  - Sequential search;
  - Direct access; and
  - File access and file organization.
- Examine other kinds of file structures in terms of
  - Abstract data models;
  - Metadata;
  - Object-oriented file access; and
  - Extensibility.
- Examine issues of portability and standardization.